Computers and Education: Artificial Intelligence 9 (2025) 100460

Contents lists available at ScienceDirect

Computers and Education: Artificial Intelligence

journal homepage: www.sciencedirect.com/journal/computers-and-education-artificial-intelligence

Investigating the impact of generative AI integration on the sustenance of higher-order thinking skills and understanding of programming logic

Jemimah Nathaniel [a,b,*], Solomon Sunday Oyelere [c,d,**], Jarkko Suhonen [a], Matti Tedre [a]

[a] School of Computing, University of Eastern Finland, FIN-80101, Joensuu, Finland
[b] Department of Computer and Information Science, Covenant University, Ota, Ogun State, Nigeria
[c] Department of Computer Science, University of Exeter, Exeter, EX4 4RN, United Kingdom
[d] Luleå University of Technology, Department of Computer Science, Electrical and Space Engineering, Sweden

ARTICLE INFO

Keywords:
Generative AI
Programming education
Higher-order thinking skills
Problem-solving
Critical thinking
Creativity
Programming logic

ABSTRACT

This study investigates how integrating generative AI (GenAI) with instructional scaffolding and prompt engineering supports higher-order thinking skills (HOTS) and programming logic. A mixed-methods design was used, combining quantitative and qualitative data. The intervention followed a one-group pretest-post-test structure over seven weeks with 25 computer science students with no prior C++ experience. The GenAI-Ped framework guided the design. It combines Bloom's taxonomy, Seelf-Regulated Learning, Universal Design for Learning, and Vygotsky's Zone of Proximal Development. Students received scaffolded support across six instructional phases, including prompt training and guided GenAI use. Quantitative results showed significant gains in problem-solving (applying constructs: $t = 2.38$, $p = 0.013$, $d = 0.475$), critical thinking (conditional reasoning: $t = 2.53$, $p = 0.018$, $d = 0.506$), creativity (applying new ideas: $t = 2.28$, $p = 0.032$, $d = 0.456$), and programming logic (loops: $t = 2.78$, $p = 0.010$, $d = 0.555$). However, smaller gains were observed in code optimization ($t = 1.693$, $p = 0.103$, $d = 0.339$) and evaluating solutions ($t = 1.732$, $p = 0.096$). Qualitative data, including feedback and GenAI chat logs, showed that prompt specificity and scaffolded feedback improved engagement, HOTS, and programming logic. The novelty of the study lies in its demonstration that the integration of GenAI into programming education using GenAI-Ped framework can sustain HOTS and programming logic while mitigating overreliance. These findings offer a practical model for integrating GenAI into programming education.

1. Introduction

Programming is a complex skill that involves logic, creativity, critical thinking, problem-solving, and attention to detail. As Lewis (2022) notes, programming extends beyond writing code to developing life skills such as resilience and analytical thinking. Due to the complexity involved in programming, traditional classrooms have embraced various technologies, tools, and innovations to enrich the teaching and learning of programming. However, educators still struggle to personalize learning despite available resources, which can make programming instruction overwhelming for educators and students (Oyelere, Suhonen, & Laine, 2017). Among the latest technologies, generative AI (GenAI) is conspicuous for its ability to transform programming education (Bermejo et al., 2024). These tools offer instant support for tasks such as coding, debugging, algorithm design, and even content generation, making programming more accessible than was previously the case (MacNeil et al., 2023; Yilmaz & Yilmaz, 2023). GenAI has been widely used in generating code (Finnie-Ansley et al., 2023; Koutcheme et al., 2023; Wermelinger, 2023), automatically generating programming exercises (Zavala & Mendoza, 2018), improving error messages (Leinonen et al., 2023), creating debugging quizzes (Logacheva et al., 2024), and automating grading (González-Calatayud et al., 2021; Ho et al., 2024). It also enables code auto-completion (Phung et al., 2023; Savelka et al., 2023a) and serves as programming assistance (Savelka et al., 2023b).

These GenAI tools are reshaping how students engage with

* Corresponding author. School of Computing, University of Eastern Finland, FIN-80101, Joensuu, Finland.
** Corresponding author. Luleå University of Technology, Department of Computer Science, Electrical and Space Engineering, Sweden
E-mail addresses: jeminath@student.uef.fi, jemimah.nathaniel@covenantuniversity.edu.ng (J. Nathaniel), s.oyelere@exeter.ac.uk, solomon.oyelere@ltu.se (S.S. Oyelere), jarkko.suhonen@uef.fi (J. Suhonen), matti.tedre@uef.fi (M. Tedre).

https://doi.org/10.1016/j.caeai.2025.100460
Received 9 April 2025; Received in revised form 6 August 2025; Accepted 6 August 2025
Available online 9 August 2025
2666-920X/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

programming concepts and how instructors facilitate learning. The practical relevance of these use cases is evident in a range of studies. For example, Papadakis (2020) found that integrating a game-development approach enhanced novice motivation and basic programming skills. Spasić et al. (2024) highlighted how teachers with no programming backgrounds can leverage GenAI prompts (for instance, "Create a memory game using JavaScript and HTML5") to generate meaningful instructional content, making GenAI a tool for expanding access to programming education. Hou et al.'s (2024) CodeTailor used GenAI to convert buggy student code into Parsons puzzles, helping students debug through structured exploration, thereby increasing engagement. Chen, Xiao, and Chen (2024) introduced CoRemix, which supports conceptual understanding by using GenAI-generated visual aids to help novices remix Scratch code. Finally, Lee et al. (2024) demonstrated that GenAI's subgoal-oriented guidance enabled learners to complete tasks more quickly than without using GenAI and with increased confidence, underscoring its value in supporting task efficiency and self-efficacy.

However, despite the benefits of GenAI tools, uncritical use risks fostering overreliance, potentially weakening understanding of programming logic and higher-order thinking skills (HOTS) such as problem-solving, critical thinking, and creativity (Dickey, Bejarano, & Chirayu, 2024;Kazemitabaar et al., 2023; Zastudil et al., 2023). Nonetheless, limited empirical research has been conducted that examines the impact of GenAI tools on sustaining HOTS and programming logic. To address this gap, the current study applies the GenAI-Ped framework (Nathaniel et al., 2025), which represents a novel approach combining GenAI tools with scaffolding strategies and prompt engineering. The GenAI-Ped framework guided the structure and implementation of the intervention. It integrates Bloom's taxonomy, Universal Design for Learning (UDL), and Vygotsky's Zone of Proximal Development (ZPD) to scaffold students' use of GenAI tools. This ensured that students were gradually supported across six instructional phases, from needs analysis to independent coding and reflection. Naeem et al. (2024) and Dickey, Bejarano, & Chirayu, 2024, have highlighted the importance of guided tasks that promote reflective thinking. Liao et al. (2024) showed that a scaffolded ChatGPT system improved computational thinking. Prather et al. (2024) cautioned that AI-generated tasks, while helpful for debugging, may widen metacognitive gaps without proper scaffolding. These studies emphasize the importance of combining GenAI tools with instructional strategies that maintain learners' skills (Obaido et al., 2023)

The overarching aim of this study is to examine how structured GenAI integration can support and sustain HOTS and programming logic in programming education. The specific objective is to evaluate the extent to which GenAI integration, supported by scaffolding and prompt engineering, enhances the development and sustenance of HOTS and programming logic in programming education. The research is guided by two research questions: (1) To what extent does GenAI integration, with scaffolding and prompt engineering, enhance the development and sustenance of HOTS in programming education? (2) How effectively does GenAI integration, with scaffolding and prompt engineering, sustain and enhance students' foundational programming logic?

This study helps educators use GenAI to enhance learning without promoting shallow engagement or overreliance on GenAI. Beyond the classroom, the intervention supports students in building essential skills for the digital workforce, aligning with broader goals like innovation, digital equity, and lifelong learning.

2. Literature review

2.1. Higher-order thinking, programming, and generative AI

HOTS include critical thinking, problem-solving, creativity, communication, and collaboration. HOTS are essential for independent learning and innovation and are globally recognized by the World Economic Forum as key competencies for the digital age (Aibin et al., 2023; Gendenjamts, 2023; Luna, 2015). These skills are essential for helping learners develop cognitive flexibility and adaptability in today's world (Astuti et al., 2019; Maryani et al., 2021). HOTS form the foundation of high-quality human capital (Misrom et al., 2020) and are often categorized into three domains: transfer (applying knowledge in varied contexts), critical thinking (analyzing and evaluating information), and problem-solving (developing practical solutions; Brookhart, 2010). The significance of HOTS has grown, particularly within global educational frameworks such as those highlighted by Zhou et al. (2023), in which competencies such as analysis, evaluation, and creativity are emphasized as crucial for navigating complex problems (Chen, Xiao, & Chen, 2024). According to Egbert et al. (2021), programming effectively fosters HOTS, particularly among K–12 learners. It encourages students to design solutions, plan logically, and think analytically. The iterative and flexible structure of programming allows students to test strategies and integrate interdisciplinary concepts (Fanchamps et al., 2021). It also generates innovative solutions, making it a fertile ground for cultivating both cognitive and creative thinking. These activities support the development of computational thinking (Falloon, 2016; Sun et al., 2022), which is a process that mirrors HOTS through abstraction, decomposition, and algorithmic reasoning.

With the advancement of educational technology, GenAI has introduced new tools for teaching and learning programming (Ahme et al., 2024), and ways to support HOTS in programming. Studies show that GenAI tools such as ChatGPT can enhance learning in introductory programming courses by offering explanations of programming concepts, handling complex topics, generating code, and supporting error correction (MacNeil et al., 2022; Deriba et al., 2023; Santos et al., 2023; Hartley et al., 2024; Ho et al., 2024). These tools also improve feedback quality and promote self-paced learning, leading to increased learner confidence and performance (Kazemitabaar et al., 2023; Gardella et al., 2024). However, although GenAI can support critical thinking (Yatigammana et al., 2023), overdependence on AI-generated solutions may hinder reflective thinking and deep engagement. This is especially true when learners rely on GenAI without understanding the underlying programming principles (Kurtz et al., 2024; Xue et al., 2024). Furthermore, since hands-on practice remains key to developing programming skills, the integration of GenAI must be guided to ensure that it supplements, rather than substitutes for, active learning (Logacheva et al., 2024). Hence, guidance is required in the integration of GenAI to sustain these skills (Dickey, Bejarano, & Chirayu, 2024).

2.2. Impact of GenAI on programming logic understanding

Programming logic is the foundation of effective coding. It includes core concepts such as loops, conditionals, functions, and control flow. This logic enables learners to translate real-world problems into algorithmic solutions, which is essential for building efficient and maintainable software. These concepts also enable learners to design solutions that are logical, efficient, and adaptable (Ibarra-Torres et al., 2024). According to Liu (2018), students who understand programming logic can better navigate both procedural and object-oriented paradigms, as they are able to think abstractly and apply concepts across different programming contexts. However, many beginners find programming logic difficult due to its abstract nature and the precision it requires (Jonsson & Tholander, 2022). These challenges can lead to frustration and disengagement. This is where GenAI tools offer unique support. GenAI has the ability to simplify syntax, generate illustrative examples, and streamline development processes (Hamadeh, 2024; Spinellis, 2024). Unlike traditional environments bound by strict syntax and limited feedback, GenAI can dynamically assist students by providing contextual suggestions and debugging support (Jonsson & Tholander, 2022). This adaptability can help learners overcome common hurdles such as syntax errors and vague semantics (Huang et al., 2023).

However, without guidance, students may become dependent on

GenAI, copying and pasting code without understanding how it works. This may result in superficial learning in which students submit correct solutions without understanding the underlying logic (Rahe & Maalej, 2025). Fernandez and Cornell (2024) refer to this as the "black-box effect," in which the code works, but the student cannot explain how or why. This problem is more pronounced when students do not receive guidance on how to interpret or critique GenAI responses.

2.3. Mapping students' programming learning with Bloom's taxonomy and pedagogical considerations

Bloom's taxonomy provides a structured framework for classifying educational objectives and cognitive processes. This makes it a valuable tool for designing programming instruction (Krathwohl, 2002; Omer, 2025). The taxonomy categorizes learning into six hierarchical levels: remembering, understanding, applying, analyzing, evaluating, and creating (Rozita et al., 2021). The six levels of the taxonomy offer a framework that allows educators to evaluate the students' knowledge and abilities as they move from simple to complex programming concepts (Rozita et al., 2021). The taxonomy also helps educators track HOTS development and adjust instruction to address learning gaps. This is particularly useful in programming in which abstract concepts often confuse beginners (Sobral, 2021). To maximize the benefits of Bloom's taxonomy, it is necessary to map programming tasks to the taxonomy. This enhances teaching effectiveness by aligning course objectives, instructional strategies, and assessments with specific cognitive levels (Cabo, 2015). Mapping Bloom taxonomy allows educators to identify gaps in students' understanding of concepts and their ability to successfully apply programming concepts (Ullah et al., 2020). Mapping has also been effective in addressing the diverse cognitive types defined within the taxonomy (Malik, 2019).

Pedagogical considerations are also vital for ensuring effective learning. These refer to the teaching methods, strategies, and conditions that influence student engagement and comprehension. According to Strawhacker et al. (2018), flexibility in lesson delivery, timely feedback, and active learning strategies all contribute to improved learning outcomes. Their study further suggests that adjusting teaching based on students' needs enhances learning in early programming education. Research by Bjursten et al. (2023) and (Sunday et al., 2020) suggests that motivation increases when students are engaged in authentic tasks. For instance, constructivist approaches encourage students to build their own understanding through exploration and reflection (Kiesler, 2022; Sullivan, 2017). Project-based learning (PBL) allows learners to apply skills in meaningful contexts (Grant, 2011). These models promote student autonomy and collaboration, and they are key factors in building problem-solving and creative thinking abilities. However, finding the right teaching method remains challenging, particularly in resource-constrained environments. Eteng et al. (2022) identified that many developing countries lack access to effective programming instruction due to limited infrastructure and trained educators. In this regard, GenAI tools offer new possibilities for supporting diverse pedagogical needs. GenAI offers numerous conventional pedagogical approaches that can improve learning experiences (Hazzan & Erez, 2024), especially in the teaching of introductory programming (Stone, 2024). However, meaningful integration requires a shift in perception, from seeing GenAI as a mere tool to embracing it as a collaborative partner in the learning process (Yan, Nakajima, & Sawada, 2024).

2.4. Theoretical aspects, scaffolding, and prompt engineering for GenAI integration in programming education

In this study, integrating GenAI into programming education is rooted in constructivist learning theories and Vygotsky's ZPD. Constructivism emphasizes that learners build knowledge through experience and interaction (Sullivan, 2017, p. 22), hence, this approach is suitable for programming education. Vygotsky's ZPD also emphasizes scaffolding, which are supportive strategies that help learners move beyond their current abilities with guidance (Erdei et al., 2017). Scaffolding describes instructional practices that help students extend their knowledge and abilities (Belland, 2017) and requires due attention to various aspects to maximize learning (Devolder et al., 2012). Scaffolding can be applied through interactive feedback, guided exercises, and collaborative learning environments, enabling learners to internalize concepts and progressively achieve independence (Sentance,Waite, & Kallia, 2019)

Integrating scaffolding techniques into GenAI chatbots provides students with a guided experience that accommodates varying levels of learner competence (Jin et al., 2024), playing an effective role in improving students' comprehension skills (Yan, Nakajima, & Sawada, 2024). For instance, integrating GenAI into high school students' education through scaffolding in a practical learning workshop has positively impacted students' understanding of GenAI concepts (Estevez et al., 2019). GenAI tools such as MindScratch showcase that adaptive scaffolding can achieve more effective learning outcomes by facilitating cognitive and creative thinking skills (Chen et al., 2025). This suggest that the constructionist prompting model for working with large language models promotes problem-solving ability through natural language programming and prompt engineering (Hsu, 2025) and advocates pedagogical practices that integrate collective project-based and active learning (Grant, 2011).

Prompt engineering is a new skill emerging from GenAI usage. It involves crafting precise prompts to receive accurate outputs (Viswanathan, 2025). The "Prompt Problems" approach encourages students to create natural language prompts that retrieve correct code from large language models. This fosters computational thinking and helps learners build the key skill of prompt design, an increasingly valuable competency in GenAI-assisted programming (Denny et al., 2023). According to Walter (2024), students' abilities to create effective prompts have the capacity to strongly shape how they interact with GenAI tools. Particularly in programming languages such as C++, prompt engineering plays a vital role in improving the effectiveness of GenAI tools (Ramona & Aleksandar, 2025). Due to the intricacies of C++ programming, such as memory management and its complicated syntax, well-structured prompts are necessary to effectively guide GenAI responses. Hence, to effectively enrich learning experiences for students, guidelines are needed (Mac, 2024). To improve the students' learning process, it is also necessary to design efficient prompts and motivate learners to identify and rectify GenAI's errors while mitigating the risks of overdependence on these tools (Dickey, Bejarano, & Chirayu, 2024).

2.5. Mapping HOTS and programming logic to Bloom's taxonomy

This study applied the GenAI-Ped framework to operationalize these mappings (Nathaniel, et.al., 2025).The framework integrates Bloom's taxonomy into both instructional design and assessment rubrics. Bloom's taxonomy comprises of six levels provides a structured way to assess and support the development of both foundational and advanced programming skills. This ensures that students' use of GenAI supports not only code generation but also HOTS development. For example, problem-solving spans understanding, applying, analyzing, and evaluating stages, as students identify problems, plan and implement solutions, and assess their effectiveness. Critical thinking is essential for debugging and decision-making. It aligns with the evaluating stage of Bloom's taxonomy, in which students justify their choices and refine code. Creativity, which involves developing original and effective solutions, maps to the creating level, encouraging students to integrate novel approaches. These criteria were selected for their relevance to programming education. They also appear in validated tools such as the Computational Thinking Scale (Gupta & Mehrotra, 2022; Korkmaz et al., 2017), which this study adapted.

3. Methods

3.1. Research model and participants

The study employed a one-group pretest-posttest design. It aimed to evaluate the impact of integrating GenAI tools, such as ChatGPT and GitHub Copilot, into programming education. The one-group pretest-posttest design was selected for both practical and ethical reasons. Ethically, it allowed all students to benefit from the intervention, avoiding an unfair control group (Campbell, Stanley, & Gage, 1963). However, because only 30 second-year computer science students volunteered to participate, with five students withdrawing due to personal reasons, the small sample size of 25 made it difficult to divide participants into groups without losing statistical power. To improve reliability, a pre-pretest was conducted before the main pretest to establish an extra baseline performance. The mixed-methods approach ensured methodological triangulation, improving the validity of findings through the convergence of quantitative and qualitative data (Creswell, 2014).

A non-probabilistic, convenience sampling method was used for the participant selection (Novosel, 2023). Participants were selected based on availability and willingness. While this method limits the generalizability of the findings, it enabled efficient access to participants within the available institutional and logistical constraints. It also allowed an in-depth exploration of the intervention within a specific educational context.

3.1.1. Reliability and validity

To ensure reliability, a pre-pretest was conducted to serve as an extra baseline before the main pretest. This step helped confirm that any changes observed in the posttest were due to the intervention rather than natural variability or repeated exposure to the test. This approach aligns with the recommendations of Campbell and Stanley (1963), who emphasize the importance of controlling for testing threats to internal validity in quasi-experimental designs. Additionally, Dimitrov and Rumrill (2003) advocate the use of repeated baseline assessments to strengthen confidence that post-intervention changes are attributable to the treatment, rather than to prior exposure. The pre-pretest thus served as a control measure, enhancing the reliability and internal validity of the study's outcomes. It was adapted from the CTS (Korkmaz et al., 2017) and Bloom's taxonomy (Krathwohl, 2002) to create a tailored HOTS scale. The new HOTS scale evaluates four key domains: problem-solving, critical thinking, creativity, and programming logic. For content validity, five experts were consulted to validate the new HOTS scale and programming exercises. Three lecturers and two industry professionals who were familiar with C++ and Bloom's taxonomy performed the grading.

3.2. Data collection tools

3.2.1. Quantitative data collection

Pretest and posttest programming exercises (Appendix 2) were used to measure students' skills before and after the intervention. These exercises were developed from the new HOTS scale, shown in Appendix 1. The rubric used for grading the pretest and posttest tasks was adapted from the multi-dimensional rubrics for analyzing free-choice Scratch programming projects (Basu, 2019) and the corresponding level of Bloom's taxonomy (Gupta & Mehrotra, 2022) to suit this research.

3.2.1.1. Rubrics. Custom rubrics were categorized into four main areas: problem-solving, critical thinking, creativity, and understanding programming logic. Each category contains specific criteria mapped to Bloom's taxonomy levels and is evaluated on a scale that ranges from Lacking Proficiency (0) to Exceeding Proficiency (4). The code produced by each student was reviewed using the rubric, and a score was assigned based on how well it met the criteria. The rubrics consist of four factors and 28 criteria, as shown in Appendix 3. For example, under "Problem Identification" in the Problem-Solving Rubric, it was necessary to check whether the student correctly identified the problem requirements, and then to determine which part of the rubric each code snippet addressed. For example, a program that sums two numbers relates to "Problem Identification" and "Applying Constructs." Student 1 wrote "sum = num1 from 2; ", which was invalid. The correct response should be "sum = num1 + num2; ". This error indicates incorrect use of operators, so in Understanding Programming Logic - > Use of Operators & Expressions, the score was Lacking (0) because it was incorrect.

As shown above, the Problem-Solving Rubric (Table 1, Appendix 3) evaluates a student's ability to address programming challenges by examining seven key areas. It begins with how effectively they identify and break down a problem, followed by their capacity to clearly define solution requirements. It then assesses their skill in planning a logical approach, applying programming constructs such as loops and conditionals, handling and correcting errors, implementing a functional solution, and, finally, testing that solution thoroughly. The Critical Thinking Rubric measures a student's ability to reason through coding challenges and refine their work. It covers error identification, in which the focus is on spotting both syntactical and logical mistakes. It then looks at debugging strategies recognizing that a student at a Developing level (2) might use basic techniques, while a student at the level of Exceeding Proficiency (4) employs more advanced methods and explains their approach, as shown in Table 2 in Appendix 3. Table 3 in Appendix 3 shows the Creativity Rubric used to assess how students introduce original ideas and innovative solutions into their programming projects. The rubric checks their ability to deliberate on different solution options, extend the problem by adding new dimensions, and integrate fresh ideas into their work. Finally, the Understanding Programming Logic Rubric in Table 4 in Appendix 3 examines a student's

Table 1
Kolmogorov-Smirnov (K-S) test result.

Category	Criterion	K-S Statistic (D)	p value
Problem-solving	Problem Identification	0.12	0.07
	Defining Requirements	0.11	0.09
	Solution Planning	0.1	0.12
	Applying Constructs	0.13	0.05
	Handling Errors	0.11	0.08
	Implementing the Solution	0.12	0.06
	Testing the Solution	0.1	0.11
Critical Thinking	Error Identification	0.11	0.08
	Debugging Strategies	0.12	0.07
	Optimizing Code	0.1	0.1
	Evaluating Solutions	0.11	0.09
	Use of Logical Constructs	0.13	0.05
	Conditional Thinking	0.12	0.06
	Application of Patterns	0.1	0.11
Creativity	Originality of Approach	0.11	0.08
	Creative Application of Constructs	0.12	0.07
	Exploring Alternatives	0.1	0.1
	Extending Problem Scope	0.11	0.09
	Application of New Ideas	0.13	0.05
	Experimentation	0.12	0.06
	Innovative Problem Approach	0.1	0.11
Understanding Programming Logic	Use of Variables	0.11	0.08
	Use of Loops	0.12	0.07
	Use of Conditionals	0.1	0.1
	Use of Operators & Expressions	0.11	0.09
	Program Initialization	0.13	0.05
	Program Termination	0.12	0.06
	Code Organization and Clarity	0.1	0.11

Table 2
Comparison of pre-pretest and pretest scores.

Category	Criterion	Pre-Pretest Mean (SD)	Pretest Mean (SD)	p value
Problem-Solving	Problem Identification	1.82 (0.61)	1.84 (0.62)	0.876
	Defining Requirements	1.35 (0.75)	1.36 (0.76)	0.912
	Solution Planning	1.67 (0.69)	1.68 (0.69)	0.943
Critical Thinking	Error Identification	1.31 (0.56)	1.32 (0.56)	0.901
	Debugging Strategies	1.11 (0.44)	1.12 (0.44)	0.889
	Conditional Thinking	1.67 (0.85)	1.68 (0.85)	0.935
Creativity	Originality of Approach	1.19 (0.50)	1.20 (0.50)	0.908
	Application of New Ideas	1.03 (0.20)	1.04 (0.20)	0.920
Programming Logic	Use of Variables	1.67 (0.94)	1.68 (0.95)	0.942
	Use of Loops	1.55 (1.04)	1.56 (1.04)	0.927
	Program Initialization	1.75 (0.97)	1.76 (0.97)	0.918

Table 3
Descriptive statistics for problem-solving skills.

Criteria	Assessment	Mean	SD
Problem Identification	pretest	1.840	0.624
	posttest	2.160	0.554
Defining Requirements	pretest	1.360	0.757
	posttest	1.600	0.816
Solution Planning	pretest	1.680	0.690
	posttest	2.000	0.707
Applying Constructs	pretest	1.520	0.653
	posttest	1.880	0.666
Handling Errors	pretest	1.200	0.500
	posttest	1.480	0.586
Implementing the Solution	pretest	1.400	0.645
	posttest	1.760	0.723
Testing the Solution	pretest	1.000	0.289
	posttest	1.200	0.500

Table 4
Paired samples *t*-test for problem-solving criteria.

Measure	t value	p value	Cohen's d
Problem Identification	2.551	0.009	0.510
Defining Requirements	1.809	0.041	0.362
Solution Planning	2.317	0.015	0.463
Applying Constructs	2.377	0.013	0.475
Handling Errors	1.899	0.035	0.380
Implementing the Solution	2.092	0.024	0.418
Testing the Solution	1.732	0.048	0.346

command of fundamental coding principles. It evaluates the effective use of variables and further assesses their skill in creating and reusing procedures and functions, using operators and expressions, properly initializing and terminating programs, organizing code clearly, and designing components that can be reused.

Each criterion is reflected in the study's adapted assessment rubric and was measured through a combination of pre-pretest, pretest, and posttest exercises. By clearly linking these constructs to Bloom's taxonomy and explicitly defining the criteria used, the study ensured theoretical coherence and practical applicability in evaluating the impact of GenAI on student learning outcomes. Each dataset collected in this study was mapped directly to specific HOTS criteria, as outlined in the assessment rubrics shown in Appendix 3.

3.2.1.2. Pre-pretest, pretest, and posttest. To the best of our knowledge, no standardized tool exists for measuring HOTS. Hence, this study adapted a hybrid version of the CTS by Korkmaz et al. (2017). The scales consist of a Likert-type rating structure with items and subfactors, and the Bloom taxonomy (Krathwohl, 2002; Chandio et al., 2016). Computational thinking is recognized as a fundamental approach for developing problem-solving skills (Nathaniel, et.al., 2025). The customized HOTS scale for programming education was created by modifying the five subfactors in the CTS, creativity, algorithmic thinking, cooperativity, critical thinking, and problem-solving, to four subfactors consisting of creativity, understanding of programming logic, critical thinking, and problem-solving. The 29 items that were collected under five factors were also modified to 18 items, as shown in Appendix 1. E-forms were used to collect the data for the pre-pretest, and all items in the instrument were assessed using a 5-point Likert scale, ranging from 1 (*strongly disagree*) to 5 (*strongly agree*).

Pretest and posttest programming exercises were obtained by converting the items in the HOTS scale, as shown in Appendix 1, to programming tasks. For instance, "I can explain how loops (for, while, do-while) work in programming" was reworded to a programming question: "*Write a program that prints all numbers from 1 to 10 using a for loop*," as shown in Appendix 2. For the pretest, students completed baseline programming tasks before the commencement of the intervention, and their solutions were graded using the rubrics to establish initial skill levels. The posttest administered after the intervention was the same as the pretest. Posttest scores were compared to pretest results to measure progress.

3.2.2. Qualitative data collection

Qualitative data was obtained through student feedback, code submission, and GenAI chat logs submitted during the intervention. At the end of each phase, students completed structured feedback forms designed to capture their experiences and perceptions of working with GenAI. This feedback included open-ended questions that encouraged learners to express how they engaged with the tools, challenges encountered, and changes in understanding over time. In addition to feedback, students submitted the GenAI chat logs or screenshots of their interactions with the GenAI (ChatGPT). These logs included the prompts students used, the GenAI-generated responses, and any modifications students made to the AI outputs before integrating them into their codes. Code submissions were analyzed to observe how students used GenAI-generated code.This combination allowed for a richer understanding of learner behavior, particularly regarding prompt use, tool interpretation, and strategic adaptation throughout the intervention phases.

3.3. Data analysis

3.3.1. Quantitative analysis

The quantitative data were analyzed using JASP version 0.19.1. Descriptive statistics, including means and standard deviations, were used to summarize overall trends in the students' performance. Inferential statistics, particularly paired samples t-tests and effect sizes (Cohen's d; Cohen, 1992), were employed to evaluate the effectiveness of the intervention. Standard deviation helped to understand the variability of the scores. Inferential statistics provides a broader view of learning trends beyond individual results. The comparison of pretest and posttest means helped determine changes in students' HOTS and understanding of programming logic following the intervention. The paired samples *t*-test was selected because it suits a one-group pre-test-posttest quasi-experimental design. This method accounts for within-subject variability and isolates the effect of the intervention by directly comparing each student's performance before and after the learning activities. To ensure the appropriateness of the statistical tests, the normality of the data was assessed using the Kolmogorov-Smirnov (K-S) test. In addition, Cronbach's alpha was used to evaluate the internal consistency of the measurement instrument. However, while

Cronbach's alpha provides insights into how well items measure a construct at a single time point, it does not address consistency across time. Therefore, test-retest reliability was used to assess the stability of the instrument and reinforce the validity of the findings.

3.3.2. Qualitative analysis

A thematic approach was used to analyze the qualitative data gathered during the intervention. Qualitative data were drawn from three sources: students feedback, GenAI-generated chat Logs, code submission and prompt logs collected during Phases 2 to 5 of the intervention. It was submitted digitally after each session using structured online forms. Thematic analysis was conducted using Braun and Clarke's (2006), it was used to examine patterns in students' experiences with GenAI tools. Student reflections and GenAI chat logs were reviewed to identify emerging themes and patterns.

Student reflections and GenAI chat logs were analyzed using NVivo 14. NVivo enabled systematic identification and organization of key themes and patterns related to the integration of GenAI tools in programming education. Prominent and representative student statements were selected to illustrate each theme. This included prompt quality, code use behavior, tool challenges, perceptions of GenAI, and the impact of scaffolding and prompt engineering. The themes were used to structure the reporting in Section 4.5, where one or more illustrative quotes per theme were included to enhance the interpretation of the findings. This process provided insight into how students engaged with the tools and how their interactions influenced learning. This approach ensured efficient data management, coding consistency, and robust theme development throughout the analysis process.

3.4. Research environment and procedure

Programming concepts were introduced using a smartboard. Students engaged in hands-on exercises using their preferred C++ Development Environment. The intervention was conducted over a period of seven weeks (twice a week), with each session lasting 2 h and 30 min. The study took place in a classroom setting and followed a structured instructional timeline. Six weeks of the study involved in-person instruction, while one week was dedicated to completing a programming project outside the classroom. PBL was integrated into the intervention to enhance learner autonomy, promote collaboration, and deepen engagement with programming concepts (Grant, 2011). The overall structure and flow of the study are summarized in Fig. 1. These visuals outline the research process used to integrate GenAI into programming education. This process is guided by the GenAI-Ped framework (Nathaniel, et.al., 2025),shown in Fig. 2: PBL provided students with opportunities to apply theoretical knowledge in practical, real-world programming tasks, culminating in a capstone project (Formative and summative assessment). Formative and summative tasks were designed to encourage independent thinking, with students using GenAI tools such as ChatGPT for code generation and debugging. Educator guidance was gradually reduced in line with ZPD and PBL's emphasis on student-driven learning (Grant, 2011). To address internet and electricity challenges, the educator provided additional internet access.

The study adopted an explanatory, sequential, mixed-methods design, consisting of two main phases (Creswell, 2014). The first phase focused on quantitative data collection through pre-pretests, pretests, and posttests to assess students' HOTS and foundational programming logic. The second phase involved qualitative data collection through GenAI chat logs, student reflections, and code submissions. The qualitative data were collected for every session. This design enabled the researchers to interpret quantitative findings in greater depth and gain a comprehensive understanding of the intervention's impact.

3.4.1. GenAI-pedagogical framework

GenAI-Ped framework in Fig. 2 was built on established pedagogical theories and consisted of six phases. It was adapted from the self-regulated learning (SRL) model by Prasad and Sane (2024), which emphasizes learner autonomy through goal setting, monitoring, and reflection. The framework incorporates a needs analysis that tailors instruction to the students' prior knowledge and accessibility requirements (Fayzulloeva & Mustafoeva, 2020) and prompt-engineering techniques from the AI-Lab framework (Dickey, Bejarano, & Chirayu, 2024; Prasad & Sane, 2024), which aid in transforming GenAI interactions into deliberate HOT exercises. Finally, Bloom's taxonomy (Krathwohl, 2002) was incorporated to ensure that learning activities and assessments move beyond basic code writing to involve analysis, evaluation, and creation, along with Universal Design for Learning (Roth et al., 2021) principles that guarantee multiple means of engagement and expression.

The implementation process of GenAI-Ped framework shown in Fig. 3, followed six structured phases designed to scaffold learning and support the development of HOTS and programming logic. Phase 1 focused on assessing students' prior knowledge and identifying learning needs (covered in weeks 1 and 2). A pre-pretest was administered to

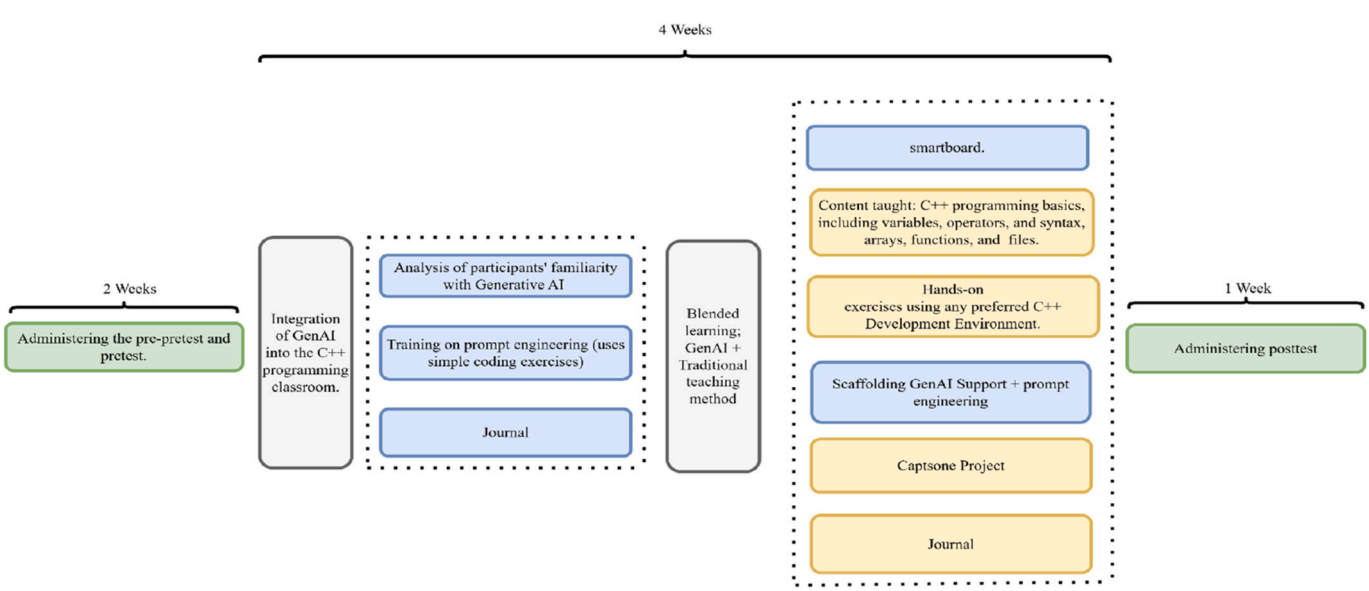

Fig. 1. Research process of integrating GenAI into programming education.

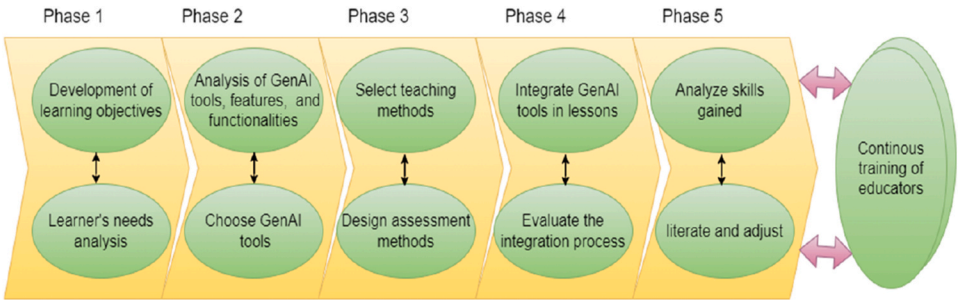

Fig. 2. The GenAI-Ped framework used in this study. It combines self-regulated learning, Bloom's taxonomy, and prompt engineering techniques to guide structured GenAI integration across six instructional phases (Nathaniel, et.al., 2025).

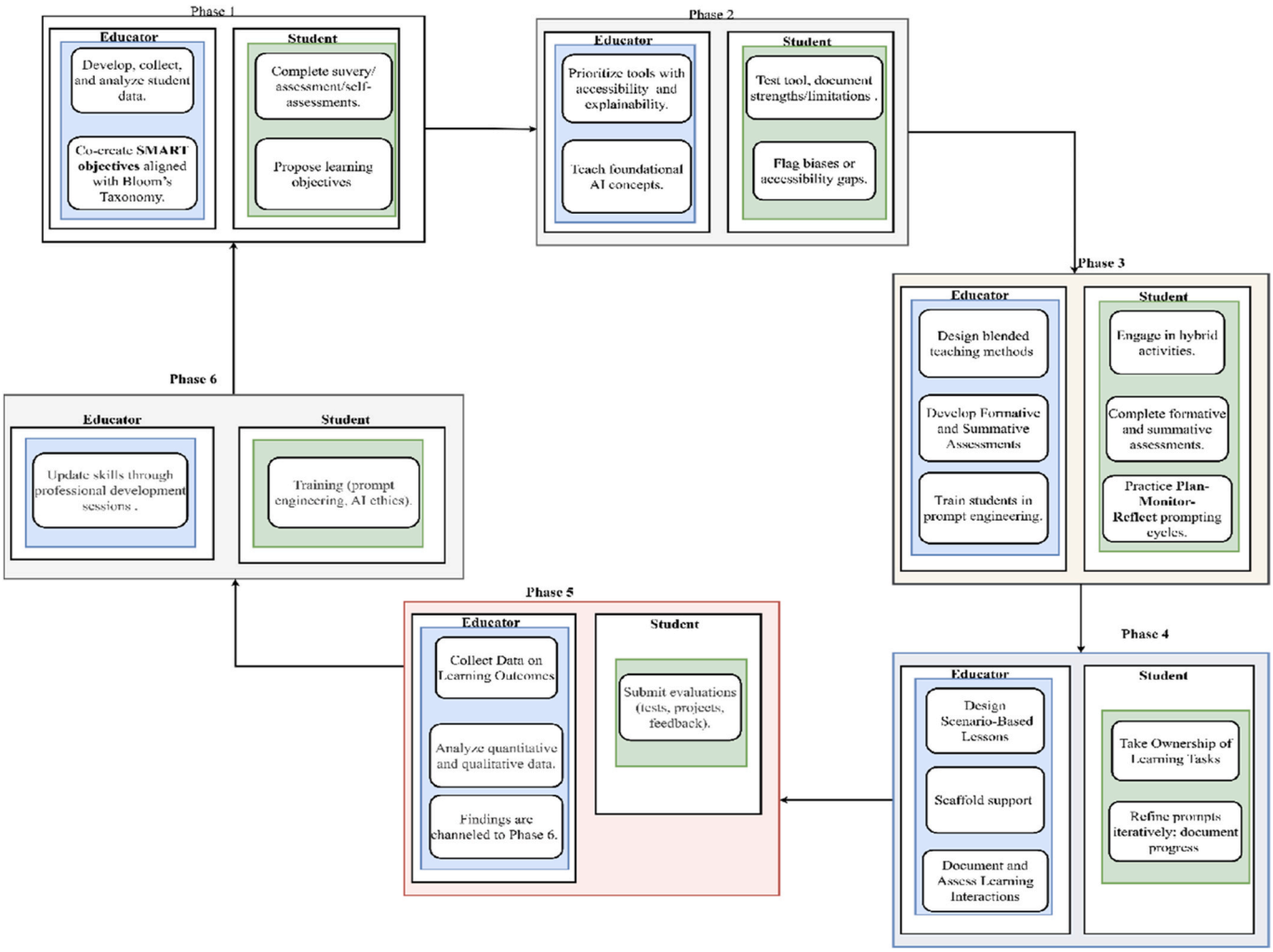

Fig. 3. Implementation process of GenAI in programming education (GenAI-Ped) framework (Nathaniel, et.al., 2025).

establish a baseline before the formal pretest. This helped eliminate test familiarity as a confounding factor.

This was followed by a formal pretest to assess students' actual starting points (students' prior knowledge). The results of the pre-pretest and pretest were compared to confirm the consistency and reliability of the measurement instrument. This is necessary to ensure that any observed learning gains can be attributed to the intervention rather than familiarity with test items.

The results confirmed that most participants were beginners. Based on these findings, the educator co-created SMART learning objectives (for instance, "Apply loops to automate tasks in C++"), which guided the weekly instructional plan. The weekly instructional topics selected included an introduction to C++, syntax rules, variables, operators, basic program structure, arrays, functions, and hands-on debugging exercises. The learning objectives were created to align with Bloom's taxonomy. In this phase students were introduced to the concept of HOTS through foundational problem-solving tasks and explored the self-regulated learning (SRL) model.

In Phase 2 (week 3), students explored and tested GenAI tools, such as ChatGPT, Gemini, Microsoft Copilot. The educator introduced GenAI by demonstrating tasks like code generation, hands-on debugging exercises and syntax explanation. Students tested tools, critique, flagged

issues (for instance, incorrect loop logic), and selected preferred tools based on their alignment with the learning objectives. The educator guided students in identifying these issues, emphasizing tools with explainable and interactive responses. The results of this showed that several GenAI tools, such as ChatGPT and GitHub Copilot were suitable for this study, however, ChatGPT was adopted due to ease of use and interactivity. Other GenAI, such as Mistral, Gemini, and Microsoft Copilot, were used when ChatGPT was down for a period during the intervention.

In Phase 3(week 3), students practiced basic prompt engineering techniques and engaged with GenAI to generate code snippets. Activities involved using the plan–monitor–reflect cycle (Prasad & Sane, 2024). This helps with refining prompts, interpreting GenAI responses, and troubleshooting inconsistencies. During the planning stage, they learned to write clear, goal-specific prompts (for instance, "Write a C++ program that reads 10 integers and outputs the even numbers"). In the monitoring stage, they tested prompts and revised them based on feedback. For example, a prompt such as "Create a loop in C++" was improved to "Create a for loop that sums even numbers from 1 to 100." During reflection, they evaluated how different prompts produced different GenAI outputs. Formative assessments during this phase included in-class quizzes (involving tasks on variables, operators, and basic program structure) and evaluations of prompt quality and code correctness. At the end of each session, students submitted both their GenAI prompt histories and resulting codes. This allows educators to assess prompt quality, code accuracy, and conceptual understanding.

Phase 3 marks the beginning of scaffolded GenAI use, as it also covered introduction to C++, syntax rules, variables, operators, basic program structure, and hands-on debugging exercises. ChatGPT was integrated into the programming lessons by using it for code generation, debugging, and explaining programming tasks. In this phase, instruction was intensely guided. During this phase, educator-led scaffolding was prominent, with live demonstrations and explanations guiding students through basic programming tasks. For instance, when GenAI produced faulty loop logic, the educator helped students revise the prompt or manually correct the logic. As students became more confident with syntax and GenAI interaction, scaffolding was reduced.

Phase 4 (weeks 4 and 5) marked a shift from guided instruction to independent learning. Students tackled intermediate concepts like loops, arrays, conditionals, functions, and hands-on debugging with minimal support. During this phase they were encouraged to write code independently before consulting GenAI. They also encourage to collaborated to detect logical errors in both GenAI-generated and peer-generated code, supporting the development of HOTS and programming logic. The educator transitioned from direct instruction to a facilitative role, offering minimal guidance only when students encountered persistent conceptual challenges in this phase. With educator's prompting reflection with questions like "What do you expect this condition to return?" Students refined their prompts and evaluated GenAI responses critically, reinforcing HOTS and understanding of programming logics. Prompt engineering was emphasized through structured practice, in which students refined their own prompts using the plan-monitor-reflect cycle. Students were instructed to formulate specific, task-relevant prompts; evaluate GenAI responses critically; and edit the code independently rather than copying the output verbatim.

In week 5 students worked independently on a creative capstone programming project using GenAI. It was designed to consolidate students' learning by applying acquired programming concepts in an independent, real-world task. The project was conducted outside the classroom to promote self-directed learning and further reduce instructional support. Students were required to build real-world applications (a library management system) using C++. The task specification included features such as adding and searching books, updating records, issuing and returning books, and generating reports. These requirements were designed to integrate arrays, functions, conditionals, loops, and file handling. To ensure accountability and promote HOTS, students were instructed to submit all GenAI-generated prompts along with the corresponding code outputs and any modifications they made. This requirement enabled the educator to evaluate not only the final code but also the reasoning behind prompt construction and how students interacted with GenAI-generated content.

Phase 5 (carried out in Week 6) focused on analyzing learning skill gains, this done by conducting posttest and comparing it with the pretest. The posttest was identical in structure and difficulty to the pretest in Phase 1 and administered to assess improvements in programming logic and HOTS. Quantitative data, including pretest and posttest scores, were evaluated using HOTS rubrics designed to assess programming tasks. These scores were then analyzed using descriptive statistics and gain score calculations to measure improvements in students' coding proficiency and HOTS. Qualitative data, gathered from reflection journals, code submission, and GenAI chat logs, were thematically analyzed using NVivo 14. These insights were synthesized by the educator to identify meaningful learning patterns and areas for improvement.

Phase 6 (weeks 7) was directly informed by the outcomes of Phase 5. This phase does not represent an endpoint but rather a practical turning point that ensures sustainability, continuous improvement, and cyclical renewal of the GenAI-Ped instructional framework in programming education. It uses the identified gaps to determine where additional training and support are needed for both students and the educator. Phase 6 involved compiling these observations and channeling them into a new set of evidence-based recommendations for future implementation. These insights guided decisions about refining instructional strategies, enhancing scaffolding, and strengthening GenAI integration. Phase 6 also laid the foundation for repeating the intervention with future cohorts, using refined instructional materials and baseline measures.

4. Findings

4.1. Reliability, validity, trustworthiness, and normal distribution

To measure the content validity, each of the earlier identified experts evaluated the HOTS scale and programming exercises on a three-point scale (essential, useful but not essential, not necessary) to determine whether the new HOTS scale aligned with Bloom's taxonomy and could be used to measure whether students had acquired the intended skills. The results indicated undisputed agreement (100 %) by the experts that the HOTS scale and programming exercises were essential in determining whether students had acquired the intended skills.

For the reliability assessment, the pretest and posttest were statistically analyzed using Cronbach's alpha to determine the internal consistency of the measurement criteria. Test-retest reliability assesses the stability of the instrument. The dataset contained 25 student records. Each record included responses to both pretest and posttest items that assess various dimensions of programming logic and HOTS. Cronbach's alpha results revealed the internal consistency of the measurement criteria was good for creativity (0.79), understanding of programming logic (0.88), critical thinking (0.82), and problem-solving (0.85). This indicates excellent internal consistency across all 28 criteria. Test-retest reliability was also calculated using Pearson correlations between pretest and posttest scores. Results showed high consistency over time: Problem-solving: 0.994–0.997 (average = 0.997), critical thinking: 0.982–0.996 (average = 0.990), creativity: 0.977–0.994 (average = 0.987), and programming logic: 0.985–0.997 (average = 0.993). These results confirmed that the instrument is both internally consistent and stable over time, addressing the key limitation of Cronbach's alpha.

Multiple strategies was used to ensure the trustworthiness of the findings. These included triangulation, using both quantitative data (test scores) and qualitative data (reflections and chat logs; Creswell & Poth, 2018; Nowell et al., 2017). An audit trail was maintained to document all procedures and ensure transparency (Korstjens & Moser, 2018). Peer debriefing sessions were also held to validate thematic interpretations

and reduce researcher bias (Birt et al., 2016; Saunders et al., 2018). Collectively, these strategies strengthened the study's credibility, dependability, and confirmability.

To determine the appropriateness of applying parametric tests, the normality of the dataset was assessed using the K-S test. As shown in Table 1, all variables across the four skill criteria, problem-solving, critical thinking, creativity, and programming logic, returned p-values above 0.05, indicating no significant deviation from a normal distribution. This finding justifies the subsequent use of paired sample t-tests.

4.2. Pre-pretest

Table 2 provides a comparative summary of the pre-pretest and pretest mean scores across all the criteria. The non-significant differences (all $p > 0.05$) corroborate that no learning gains occurred between the pre-pretest and pretest phases, thereby confirming that students' baseline skills remained stable. This control step is critical in attributing any subsequent posttest improvements to the intervention itself, rather than to test familiarity or incidental exposure. It strengthens the internal validity of the study and sets a clear benchmark for evaluating the impact of GenAI integration.

The lack of significant differences between pre-pretest and pretest scores confirms the stability of the initial skill levels. This reinforces the validity of posttest improvements as genuine outcomes of the intervention. This design strengthens the internal validity of the study by ruling out practice effects or spontaneous skill development unrelated to the GenAI integration.

4.3. GenAI integration and higher-order thinking skills

4.3.1. Findings on problem-solving skills

The descriptive results in Table 3 show a consistent upward trend across all seven problem-solving criteria, with notable increases in mean scores for "Problem Identification" (from 1.840 to 2.160) and "Solution Planning" (from 1.680 to 2.000). These improvements reflect not only higher confidence in understanding problems but also in structuring viable solutions. It shows that students moved from being passive recipients of instruction to active problem-solvers, supported by scaffolded GenAI use.

As reported in Table 4, statistically significant improvements ($p < 0.05$) were observed across all seven sub-criteria. Particularly strong effects were seen in "Problem Identification" ($t = 2.551$, $p = 0.009$, $d = 0.510$) and "Applying Constructs" ($t = 2.377$, $p = 0.013$, $d = 0.475$); thus, highlighting the role of prompt engineering in guiding students to dissect and tackle programming challenges logically. These results suggest that GenAI tools, when structured with educator scaffolding, can enhance not just procedural knowledge but deeper cognitive engagement with programming tasks.

Table 5
Descriptive statistics for critical thinking criteria.

Criteria	Assessment	Mean	SD
Error Identification	pretest	1.320	0.557
	posttest	1.640	0.700
Debugging Strategies	pretest	1.120	0.440
	posttest	1.320	0.627
Optimizing Code	pretest	1.200	0.500
	posttest	1.360	0.569
Evaluating Solutions	pretest	1.040	0.455
	posttest	1.240	0.523
Use of Logical Constructs	pretest	1.560	0.651
	posttest	1.920	0.759
Conditional Thinking	pretest	1.680	0.852
	posttest	2.120	0.726
Application of Patterns	pretest	1.560	0.821
	posttest	1.520	0.872

Table 6
Paired samples t-test for critical thinking criteria.

Measure	t value	p value	Cohen's d
Error Identification	2.138	0.043	0.428
Debugging Strategies	2.000	0.057	0.400
Optimizing Code	1.693	0.103	0.339
Evaluating Solutions	1.732	0.096	0.346
Use of Logical Constructs	2.377	0.026	0.475
Conditional Thinking	2.529	0.018	0.506
Application of Patterns	0.171	0.866	0.034

Table 7
Descriptive statistics for creativity criteria.

Criteria	Assessment	Mean	SD
Originality of Approach	pretest	1.200	0.500
	posttest	1.400	0.577
Creative Application of Constructs	pretest	1.120	0.881
	posttest	1.600	0.816
Exploring Alternatives	pretest	1.120	0.332
	posttest	1.400	0.500
Extending Problem Scope	pretest	1.040	0.200
	posttest	1.240	0.436
Application of New Ideas	pretest	1.040	0.200
	posttest	1.320	0.557
Experimentation	pretest	1.280	0.614
	posttest	1.640	0.700
Innovative Problem Approach	pretest	0.920	0.493
	posttest	1.080	0.400

4.3.2. Findings on critical thinking

Table 5 reveals substantial gains in critical thinking components, particularly "Conditional Thinking," which rose from 1.680 to 2.120, and "Use of Logical Constructs" rising from 1.560 to 1.920. These findings suggest that students increasingly adopted logical reasoning and structured thinking in their approach to coding, a shift likely to have been prompted by GenAI feedback and SRL cycles. Such growth reflects an internalization of programming logic beyond rote application.

According to Table 6, critical thinking sub-criteria such as "Conditional Thinking" ($t = 2.529$, $p = 0.018$, $d = 0.506$) and "Use of Logical Constructs" ($t = 2.377$, $p = 0.026$, $d = 0.475$) demonstrated statistically significant and moderate gains. However, "Optimizing Code" did not improve significantly ($t = 1.693$, $p = 0.103$). This suggests that students progressed in logic but continued to struggle with code efficiency and performance tuning. This gap guides future interventions that incorporate GenAI prompts tailored to performance optimization.

4.3.3. Findings on creativity

Creativity measures in Table 7 indicate improved student originality and experimentation. For instance, "Creative Application of Constructs" increased from 1.120 to 1.600, and "Experimentation" from 1.280 to 1.640. The effect sizes for the improvements ranged from 0.456 to 0.517, suggesting moderate, but statistically significant, improvements. These results suggest that GenAI tools did not suppress creativity through automation but rather enabled students to explore multiple solutions, supported by real-time feedback. This aligns with constructivist theories in which creativity flourishes through iterative design and reflective thinking.

As shown in Table 8, key improvements were evident in "Exploring Alternatives" ($t = 2.585$, $p = 0.016$, $d = 0.517$) and "Experimentation" ($t = 2.571$, $p = 0.017$, $d = 0.514$), both with moderate effect sizes. These gains imply that the students became more open to trial-and-error learning, which is likely to have been encouraged by GenAI's low-risk environment for experimenting with new ideas. However, less change was observed in "Innovative Problem Approach," indicating the need for further support to push students beyond functional solutions to highly original innovations.

Table 8
Paired samples *t*-test for creativity criteria.

Measure	t value	p value	Cohen's d
Originality of Approach	1.732	0.096	0.346
Creative Application of Constructs	2.493	0.020	0.499
Exploring Alternatives	2.585	0.016	0.517
Extending Problem Scope	2.000	0.057	0.400
Application of New Ideas	2.281	0.032	0.456
Experimentation	2.571	0.017	0.514
Innovative Problem Approach	1.693	0.103	0.339

Table 9
Descriptive statistics for understanding programming logic criteria.

Criteria	Assessment	Mean	SD
Use of Variables	pretest	1.680	0.945
	posttest	2.200	0.816
Use of Loops	pretest	1.560	1.044
	posttest	2.160	0.850
Use of Conditionals	pretest	1.640	0.952
	posttest	2.160	0.898
Procedures & Functions	pretest	1.120	0.526
	posttest	1.320	0.476
Use of Operators & Expressions	pretest	1.680	0.852
	posttest	2.280	0.843
Program Initialization	pretest	1.760	0.970
	posttest	2.360	0.757
Program Termination	pretest	1.120	0.666
	posttest	1.360	0.490
Code Organization and Clarity	pretest	0.880	0.726
	posttest	1.240	0.779
Reusability of Code Components	pretest	0.600	0.500
	posttest	0.880	0.332

Table 10
Paired samples *t*-test for understanding of programming logic criteria.

Measure	t value	p value	Cohen's d
Use of Variables	2.831	0.009	0.566
Use of Loops	2.777	0.010	0.555
Use of Conditionals	2.316	0.029	0.463
Procedures & Functions	1.732	0.096	0.346
Use of Operators & Expressions	2.882	0.008	0.576
Program Initialization	3.286	0.003	0.657
Program Termination	1.659	0.110	0.332
Code Organization and Clarity	2.092	0.047	0.418
Reusability of Code Components	3.055	0.005	0.611

4.4. GenAI integration and foundational programming logic

The descriptive analysis in Table 9 reveals consistent improvement in foundational programming concepts. "Use of Variables" rose from 1.680 to 2.200, while "Program Initialization" rose from 1.760 to 2.360. These results highlight how the structured intervention not only improved conceptual knowledge but also students' fluency in syntax and control flow. The data underscore the importance of integrating GenAI within scaffolded instruction to support novice programmers in mastering fundamental logic.

The *t*-test results in Table 10 show statistically significant gains across most logic areas, with the highest effect size observed in "Program Initialization" ($t = 3.286$, $p = 0.003$, Cohen's $d = 0.657$; Cohen, 1992). This suggests that GenAI tools helped students understand the procedural flow of a program, including setup and data structure planning. Marginal gains in "Procedures & Functions" and "Program Termination" imply that students may still require targeted instruction in managing modularity and lifecycle design in code. In areas where AI may be less intuitive without expert guidance.

4.5. Thematic approach to qualitative data on generative AI in programming education

A thematic analysis of student reflections and GenAI logs revealed recurring themes: (1) prompt quality, (2) code submission and integration of GenAI-generated code, (3) challenges in using GenAI tools, (4) students' perceptions of GenAI tools, and (5) the role of scaffolding and prompt engineering as learning supports. These themes provided contextual depth to the quantitative results and helped explain patterns in HOTS and programming logic gains.

4.5.1. Prompt quality: vague vs. specific prompts

The study by Walter (2024) emphasizes that prompts influence how students interact with GenAI tools, making prompt literacy essential in programming education. To analyze the level of influence, the chat logs were segmented to understand the distinct interactions, with each representing a student's prompt and the corresponding GenAI-generated response. These interactions were then coded based on the complexity of prompts (for instance, vague vs. specific). The findings indicate that students who wrote specific and detailed prompts (see Fig. 4a) showed better engagement and learning, while those who used vague prompts produced weaker outcomes (see Fig. 4b). This suggests that the use of specific prompts enabled students to receive more accurate and relevant responses from GenAI, which in turn supported a deeper understanding of programming concepts. This finding aligns with the research of Denny et al. (2023), which argues that well-crafted prompts facilitate more meaningful AI interaction and foster computational thinking. In contrast, vague prompts often led to generic or less useful responses, limiting the students' ability to learn effectively.

4.5.2. Code submission: integration of GenAI-Generated code

This finding shows that some participants still directly copied and pasted the code obtained from the GenAI, stating "*it is too lengthy to read, so I just copy and paste the answer directly*," while another student admitted, "*Sometimes the AI's code was so complex that I didn't even try to understand it. I just submitted it and hoped for the best.*" Copying and pasting GenAI-generated code was common early in the intervention. This was when students were still unfamiliar with GenAI and opted to use responses without editing or understanding them. While copying code without understanding helped students' complete tasks quickly. This behavior reflects a common risk: treating GenAI as a shortcut rather than a learning partner, as noted by Rahe and Maalej (2025).

As one student reflected, "*At first, I just copied and pasted the code from ChatGPT because it gave me quick answers. But later, I realized I wasn't learning, I was just getting the assignment done.*" This reflection aligns with Fernandez and Cornell (2024), who warn of "black-box" programming in which reliance on AI obscures the underlying comprehension of the logic. The findings reveal that students who relied heavily on copying GenAI-generated code showed less improvement in areas such as code optimization and critical evaluation of solutions. Students who actively edited, tested, and debugged GenAI-generated code demonstrated a deeper understanding of programming concepts and showed clear improvement in HOTS and understanding of programming logic. As one of these students remarked, "*The AI's solution had a bug, and fixing it taught me more than if I'd written the code from scratch.*" Another added, "*I asked ChatGPT to explain its logic, then rewrite it in my own way. That's when things really clicked.*"

4.5.3. Challenges in using GenAI tools

Despite the benefits of GenAI tools in programming education, students encountered several challenges during the intervention. One major issue was infrastructure limitations. Unstable internet and electricity disrupted access to GenAI tools, particularly during hands-on coding sessions and assignment submissions. These challenges are consistent with Eteng et al. (2022), who noted that infrastructural deficits impair equitable access to technology-enhanced learning. Another

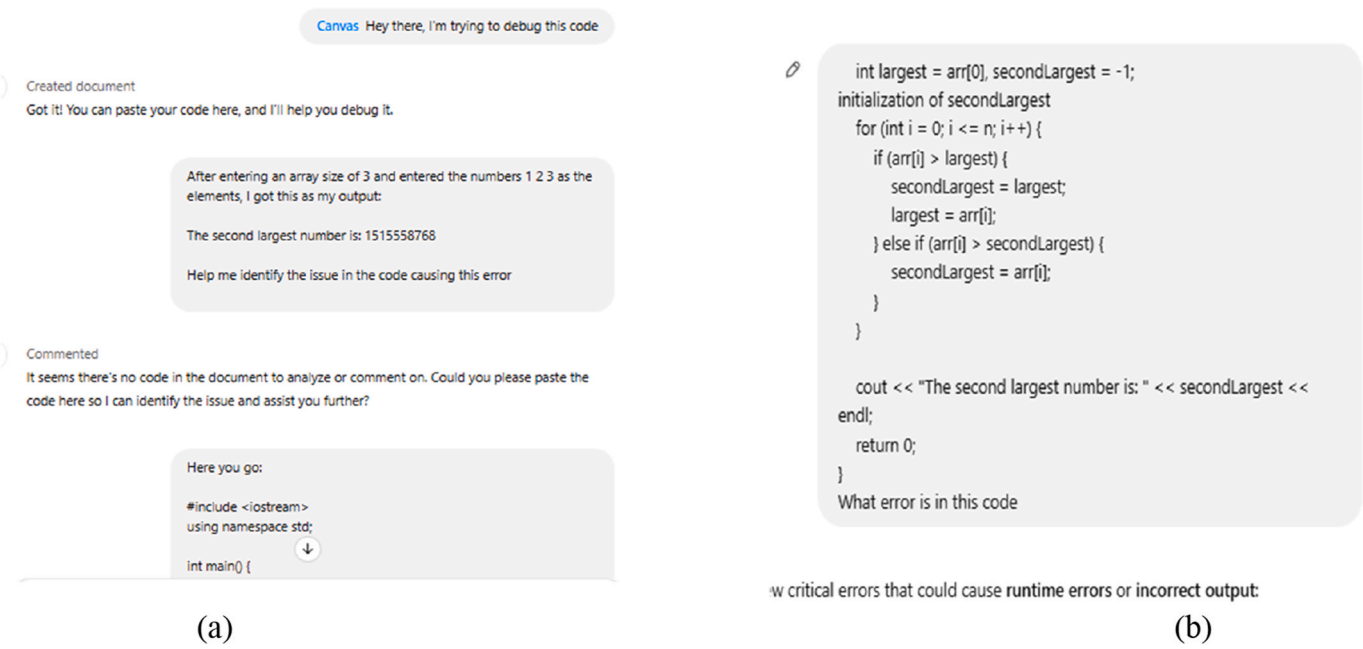

Fig. 4. Snippet of GenAI chat.

significant challenge was the misinterpretation of GenAI-generated responses. Students tend to misinterpret GenAI responses due to poorly crafted prompts, leading to errors in their code or misinterpretation of programming concepts. Misinterpretation of GenAI output echoes findings by Shanto et al. (2024), who noted that novice programmers often misread AI explanations, leading to faulty implementations. Several students also expressed concern about becoming overly reliant on GenAI, stating, "*Sometimes I feel like I lean too much on the AI instead of figuring it out myself.*" This reflects the risk noted by Kazemitabaar et al. (2023) and Naeem et al. (2024), who emphasize the possibility of diminished cognitive engagement when AI tools are used without structured guidance.

Additionally, students noted that the ease with which they could obtain answers made it tempting to skip the problem-solving process altogether. Raising concerns about evaluating students' actual programming abilities. As previously noted, Fernandez and Cornell (2024) cautioned against this "black-box" learning in which students use functional code generated by AI without understanding how it works. These challenges affected the effectiveness of the tools and the overall learning experience, highlighting the need for robust infrastructure and structured guidance to ensure responsible and effective use of GenAI tools in education.

4.5.4. Students' reflections: perceptions of GenAI tools

Students shared a range of reflections on their experiences of using GenAI tools during the intervention. Overall, most expressed positive perceptions, highlighting the tools' usefulness in improving their understanding of programming logic and reducing frustration with coding tasks. One student stated that "*GenAI can be likened to a textbook but in a more summarized and digital presentation, as each student can move at their own* pace*, as integrating GenAI into programming makes learning programming less overwhelming.*" because GenAI can provide quick and accessible explanations. Students' perceptions of GenAI as a "digital textbook" resonate with the findings of Yan, Nakajima, and Sawada (2024), who described GenAI as a cognitive support that personalizes learning pace and feedback. Many described GenAI as a helpful "coding assistant" that provided instant feedback, clarified errors, and simplified complex concepts only when prompted correctly. These reflections are consistent with studies by Ahme et al. (2024) and MacNeil et al. (2022), who found that students using GenAI tools gained confidence and performed better in programming tasks than without Gen AI due to timely and relevant support.

Most students who volunteered for this intervention preferred ChatGPT. They found it easier to use because of its interface and the option to share chat logs for collaboration. However, the participants expressed concerns that the ease of accessing GenAI-generated solutions might lead to a lack of genuine learning and participation. With one participant stating, "*it becomes harder to evaluate students actual coding abilities when GenAI assistance is involved and it should not be incorporated for beginners but for people with general understanding of programming, as any misinterpretation of prompts could affect the performances of GenAI.*" This quotation emphasizes the importance of personal practice and the need for structured guidance on effectively using GenAI tools. Without a solid understanding of syntax and logic, users may not fully comprehend the underlying principles of the code they have generated (Fernandez & Cornell, 2024; Spinellis, 2024). This dual perception aligns with Alves and Cipriano (2024), who observed that while GenAI tools support confidence and task completion, they may hinder deep learning if introduced too early or without guidance.

4.5.5. Scaffolding and prompt engineering as learning supports

Throughout the intervention, scaffolding and prompt engineering emerged as essential supports for students learning to effectively use GenAI tools in programming. Many students initially lacked the skills to frame precise prompts or interpret AI-generated code. However, with structured guidance, they gradually learned how to interact more critically and purposefully with GenAI tools. This finding aligns with the principles of Vygotsky's ZPD, which emphasizes the importance of guided support in helping learners achieve tasks just beyond their independent capabilities (Erdei et al., 2017). Scaffolding was provided through demonstrations, step-by-step instructions, and reflective prompts. These supports were gradually reduced as students became more confident in writing prompts and evaluating GenAI responses. This process mirrors the approach recommended by Belland (2017), who argued that lessening support over time promotes student autonomy and deeper learning. By scaffolding both the programming content and the use of GenAI, learners developed not only technical skills but also metacognitive awareness.

Prompt engineering was also emphasized as a skill in itself. Students were taught how to structure clear, goal-oriented prompts from the SRL

model (Prasad & Sane, 2024). Several students reflected that these supports helped them transition from passive tool users to active problem-solvers. One student noted, "*When I got something wrong and prompted following the SRL technique, the AI didn't just give me the answer but showed me the logic gap in my reasoning.*" This shift aligns with Denny et al. (2023), who demonstrated that prompt engineering enhances computational thinking by requiring learners to deconstruct problems and reason through language.

5. Discussion and implications

5.1. To what extent does GenAI integration, with scaffolding and prompt engineering, enhance the development and sustenance of HOTS in programming education?

The study found that integrating GenAI tools into programming education, alongside scaffolding and prompt engineering, supported students in developing and sustaining HOTS. These skills help them analyze problems, evaluate possible approaches, and design original solutions instead of relying on memorized procedures (Brookhart, 2010; Egbert et al., 2021). Quantitative findings revealed improvements in six of the seven measured problem-solving criteria. Notably, problem identification and applying constructs showed the highest effect sizes ($d = 0.510$ and $d = 0.475$, respectively), indicating that GenAI tools were particularly effective in guiding students through deconstructing problems and applying coding constructs. These findings support earlier research that shows GenAI tools can enhance problem-solving more effectively than traditional programming instruction (Min & Heng, 2024). This is because it makes error messages clearer and simplify complex concepts (Kimmel et al., 2024; Poitras et al., 2024). The tools also helped lower barriers to entry and introduced productive challenges that encouraged deeper engagement and more creative solutions (Jonsson & Tholander, 2022).

The results show that GenAI tools support some aspects of critical thinking. This is evident in the observed gains in error identification, conditional thinking (1.680–2.120), and logical constructs (1.560–1.920). However, GenAI tools did not improve code optimization or solution evaluation. This suggests that GenAI may not support all critical thinking sub-criteria equally. This may be because the accuracy of GenAI-generated content impacts students' critical thinking (Schefer-Wenzl et al., 2025). The marginal improvements in debugging strategies and evaluating solutions suggest that these skills may require additional instructional support or alternative strategies to achieve significant gains. Structured guidance and targeted exercises can help students build critical thinking skills when using GenAI (Alves & Cipriano, 2024), while structured learning activities incorporating GenAI can foster a more reflective approach to programming tasks (Naeem et al., 2024).

Certain sub-criteria in the creativity criteria also improved after the GenAI integration into programming education. For instance, students who used GenAI tools to explore alternative solutions or apply new ideas demonstrated a greater ability to think outside the box and develop unique programming solutions. This finding aligns with Zviel-Girshin's (2024) research suggesting that GenAI tools have the potential to foster creativity by providing adaptive support and encouraging exploration. While Sauvola et al. (2024) highlights that GenAI helps automate routine tasks and promote new forms of creativity in software design, this study found limited gains in originality and innovation. This suggests GenAI may not equally support all creativity sub-criteria. This emphasizes that the need for a balance between utilizing GenAI for efficiency and maintaining a creative edge is a critical consideration for software practitioners (Jackson et al., 2024).

Further, the qualitative findings added depth to the quantitative results and provided insights into how students experienced the learning process. The findings revealed that using GenAI helped students explore alternative solutions, develop deeper HOTS, and remain engaged with programming tasks. However, these benefits were most evident when GenAI was used as a thinking partner, not just as a shortcut to answers. This finding aligns with observations by Jin et al. (2024), who found that integrating scaffolding techniques in GenAI provides a guided experience, and well-crafted prompts significantly influence the way that students interact with GenAI tools (Walter, 2024). Additionally, the incorporation of scaffolding techniques and prompt engineering were able to increase collaboration among the participants. This plays a major role in shaping students' experience with GenAI tools and HOTS development.

5.2. How effectively does GenAI integration, with scaffolding and prompt engineering, sustain and enhance students' foundational programming logic?

The results of this study show that the integration of GenAI, supported by scaffolding and prompt engineering, had a positive effect on students' understanding and retention of programming logic. This was evident in the gains across most sub-criteria in the programming logic criteria. The results show that by the end of the intervention, students who had struggled with control structures could construct logical sequences and debug errors more accurately. This is because the structured use of GenAI allowed them to test their ideas and receive immediate, relevant feedback. Highlighting that GenAI's timely support during programming (Ghimire & Edwards, 2024) appears to aid understanding. This improvement is consistent with findings by Egbert et al. (2021), who noted that programming logic develops more effectively when students are engaged in active problem-solving rather than passive code consumption. However, minimal gains in procedures, functions, and program termination sub-criteria suggest these areas may require different instructional strategies, as GenAI tools such as ChatGPT face challenges in addressing complex student confusions and adapting responses to individual needs, which are crucial for fostering a deeper understanding of programming logic (Shanto et al., 2024).

Notably, the qualitative responses reflected that students who actively edited and tested GenAI-generated code demonstrated a better understanding of programming logic, while those who copied code passively recorded negligible gains. This finding explains the quantitative results indicating marginal increments in areas such as procedures and functions and program termination. Additionally, the GenAI chat log and code submission analysis indicated that students who used GenAI tools to explore programming concepts (for instance, variables, loops, conditionals) and debug their code greatly enhanced their understanding of programming logic. This aligns with the quantitative findings, which show improvements in core concepts such as variable application ($t = 2.831$, $p = 0.009$) and loop implementation ($t = 2.777$, $p = 0.010$). Several students also noted that creating precise prompts forced them to break down problems before asking for help, which improved their logical reasoning.

These improvements suggest that integrating GenAI into programming education, particularly within a structured framework such as GenAI-Ped, effectively supported some HOTS and the programming logic development of the learners. The intervention was designed to gradually shift responsibility from the teacher to the learner, which contributed to improved performances. These results align with previous studies such as that by Kazemitabaar et al. (2023), in which guided use of AI led to higher learning gains. However, the novelty of this study lies in its practical demonstration of how GenAI, when paired with instructional scaffolding and prompt engineering, can be used not only to support but also to sustain some HOTS and programming logic criteria over time. However, the relatively small sample size of 25 students poses constraints concerning the breadth of these conclusions, limiting the extent to which the results can be generalized to larger or more diverse populations. Despite this, the study offers valuable initial evidence that incorporating GenAI tools thoughtfully into programming education can support sustained learning gains and pedagogical innovation.

5.3. Theoretical and practical implications

This study presents both theoretical and practical implications for programming education in the era of GenAI. Theoretically, the findings support the application of constructivist learning theory (Sullivan, 2017, p. 22) and ZPD (Shabani et al., 2010). These theories suggest that learners build knowledge more effectively when they receive appropriate support during challenging tasks. The use of scaffolding (Belland, 2017) and prompt engineering (Dickey, Bejarano, & Chirayu, 2024) in this study helped students engage in programming tasks in a way that sustained HOTS and programming logic. The study also contributes to the emerging conversation around the pedagogical use of GenAI in education and highlights the need to move beyond viewing GenAI tools as shortcuts or answer generators. Instead, they should be seen as learning partners that, when combined with reflective tasks and structured support, can promote deep thinking and skill development. The findings offer a practical model for integrating GenAI into programming classrooms without reducing learners' engagement with core concepts.

Practically, the results suggest programming educators should introduce GenAI tools alongside well-defined instructional strategies. Students need training on how to craft prompts, evaluate AI responses, and reflect on what they learn. In the absence of structured guidance, students risk engaging in surface-level learning or becoming over-dependent on GenAI tools.The plan-monitor-reflect technique of prompting by Prasad and Sane (2024) offers a useful structure for guiding student interaction with GenAI. This approach encourages students to think more clearly about problems and engage more deeply with the logic of their codes. Educators should also consider varying levels of scaffolding depending on students' prior experience. Beginners may need more guided examples and support, while more advanced learners could benefit from open-ended tasks that challenge their HOTS and logic.

5.4. Ethics

This study involved human participants specifically, second-year undergraduate students. Before the intervention began, all participants were informed about the purpose of the research, their role in it, and their right to withdraw at any time without penalty. Informed consent was obtained from all 25 students in writing.

This study received ethical approval from Covenant University Health Research Ethics Committee, accredited by the U.S department of Health & Human services with registration number IOG0010037 and IOG0010037. National Health Research Ethics Committee under the registration number NHREC/CU-HREC/1/01/2-25.The assigned protocol number is CHREC/1097/2025. All research procedure were conducted in accordance with the approved protocol and ethical standard to ensure the protection of participants' right, confidentiality, and well-being.

Confidentiality was maintained throughout the study. No names or personal identifiers were collected. Data were stored securely and used only for research purposes. Students were told that their grades and participation in the study would not affect their academic standing in any way.

5.5. Limitations and future research

Despite the study's positive findings, overreliance and academic integrity issues arose in student responses, as scaffolding and prompt engineering may not have been applied outside the classroom, concealing actual skill development. This aligns with broader ethical debates regarding the promotion of clear policies on GenAI usage to maintain academic integrity and skill authenticity (Akgun & Greenhow, 2022; Holmes et al., 2022). Used as a strategy, integration of GenAI will help avoid undermining students' independent learning (Naeem et al., 2024), requiring a clear need for activities that promote collaboration in the GenAI curriculum (Sunday et al., 2020). Nonetheless, over-dependence on GenAI tools can detract from the acquisition of HOTS and understandings of programming logic.

Practical challenges were also encountered during the intervention, including the sample size, internet reliability issues, power supply disruptions, and language complexity, which occasionally hindered the effective use of GenAI tools. Study limitations included a small sample size, a single-institution context, and reliance on self-reported reflections. The modest sample size of 25 participants constrains the degree to which this study can be generalized to broader student populations or different educational settings. Future studies should plan to use larger, sample sizes to investigate how to address these barriers when integrating GenAI tools into programming education.

6. Conclusion

This study shows that integrating GenAI through a structured pedagogical framework can support the development and retention of HOTS and programming logic. The GenAI-Ped framework supported scaffolded tool use, prompt engineering, and independent learning over time.

The results indicate that GenAI tools alone are not sufficient to promote deep learning. Their impact depends on the presence of scaffolded support, well-defined instructional goals, and structured prompting strategies. The use of prompt engineering helped students learn to formulate precise queries, reflect on AI responses, and revise their code accordingly. These practices shifted students away from passive consumption of GenAI outputs and encouraged metacognitive awareness and purposeful problem-solving.

In practice, this study offers a replicable model for responsible GenAI integration in programming education. Educators should combine GenAI tools with pedagogical frameworks that support gradual release of responsibility and align learning tasks with cognitive objectives. The GenAI-Ped framework used in this intervention allowed students to not only complete programming tasks but to internalize underlying logic and reasoning processes. This approach helped mitigate the risk of overreliance and preserved the authenticity of skill development.

CRediT authorship contribution statement

Jemimah Nathaniel: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Solomon Sunday Oyelere:** Writing – review & editing, Validation, Supervision, Methodology, Investigation, Conceptualization. **Jarkko Suhonen:** Writing – review & editing, Validation, Supervision. **Matti Tedre:** Writing – review & editing, Validation, Supervision.

Availability of data and material

To protect participant privacy, the raw data from this study (including chat logs and assessments) are not publicly shared. However, anonymized excerpts or the assessment rubric can be provided upon request to support further research.

Ethics approval statement

Informed consent was secured from all participants, with explicit clarification that participation was voluntary and withdrawal was permitted at any stage. To protect participant anonymity, all data were de-identified during analysis, and pseudonyms were used in qualitative excerpts.

Funding

This work does not receive any financial support.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Not Applicable.

Appendix 1

Creativity

I can develop unique solutions to programming problems.
I enjoy brainstorming multiple ways to solve a problem.
I can use coding to create something new or innovative.
I feel confident exploring different programming tools to improve my work.

Critical Thinking

I can evaluate different approaches and choose the best one to solve a problem.
I analyze problems deeply before starting to code.
I enjoy troubleshooting and debugging errors in programs.
I can recognize patterns and connections in programming tasks.
I feel confident questioning and refining my programming solutions.

Problem-Solving

I can break down complex programming problems into smaller, manageable parts.
I feel confident identifying the main issues in a problem.
I can persevere through challenges when coding.
I can apply learned concepts to solve new or unfamiliar problems.

Understanding Programming Logic

I understand the use of conditional statements (For example, if-else, switch) in programming.
I can identify and fix logical errors in a program.
I can explain how loops (For example, for, while, do-while) work in programming.
I understand the purpose of variables and how to use them effectively.
I feel confident designing and implementing logic for small programming tasks.

Appendix 2

Creativity

Write two different programs to find the largest number in an array.
Create a program that generates a random password of 8 characters, including letters, numbers, and symbols.
Write a program that uses a library to calculate the square root of a number.
Design a program that simulates a simple calculator with addition, subtraction, multiplication, and division.

Critical Thinking

Compare two sorting algorithms and explain which one is more efficient.
Write a program that checks if a number is prime and explain the logic behind your solution.
Debug the following program that is supposed to calculate the average of three numbers but contains errors.
Write a program that prints the Fibonacci sequence up to the 10th term and explain the pattern.
Write a program to reverse a string and then optimize it to use fewer lines of code.

Problem-Solving

Write a program that calculates the sum of all even numbers between 1 and 100.
Write a program that identifies and prints all duplicate elements in an array.
Write a program that converts a decimal number to binary without using built-in functions.
Write a program that calculates the total cost of items in a shopping cart, including tax.

Understanding Programming Logic

Write a program that checks if a number is positive, negative, or zero using conditional statements.
Write a program that prints all numbers from 1 to 10 using a for loop.
Write a program that swaps the values of two variables without using a temporary variable.
Predict the output of the following program and explain the flow of execution.

Appendix 3

Table 1
Problem-Solving Rubric

Problem-solving						
(Assessing the ability to identify, analyze, and implement solutions for programming challenges)						
Criteria	Bloom's Taxonomy Level	Lacking Proficiency (0)	Emerging (1)	Developing (2)	Proficient (3)	Exceeding Proficiency (4)
Problem Identification	Understanding	Does not identify or misinterprets the problem.	Identifies basic elements but misses key parts.	Identifies most aspects with minor gaps.	Clearly identifies and understands all aspects of the problem.	Breaks down complex problems into smaller, solvable parts.
Defining Requirements	Understanding	Cannot define the problem requirements.	Defines only vague or partial requirements.	Defines basic requirements with some detail.	Defines clear and detailed requirements for the solution.	Defines comprehensive requirements, anticipating challenges and constraints.
Solution Planning	Applying	No plan for solving the problem.	Develops a very basic or incomplete plan.	Creates a functional plan addressing key step.	Develops a structured, detailed plan for solving the problem.	Develops an advanced, well-structured plan incorporating optimization strategies.
Applying Constructs	Applying	Fails to use constructs relevant to the problem.	Uses constructs inconsistently or incorrectly.	Uses appropriate constructs to address key parts of the problem.	Uses constructs effectively to solve the problem comprehensively.	Combines constructs innovatively to solve the problem in a unique or advanced way.
Handling Errors	Applying → Evaluating	Ignores errors or fails to recognize them.	Identifies errors but struggles to address them.	Fixes basic errors but misses more complex ones.	Effectively identifies and resolves all errors.	Anticipates, identifies, and resolves errors systematically and efficiently.
Implementing the Solution	Applying	Cannot implement a working solution.	Implements a partial or incomplete solution.	Implements a functional solution with minor issues.	Implements a fully functional and reliable solution.	Implements an optimized, scalable, and advanced solution.
Testing the Solution	Analyzing → Evaluating	Does not test or tests minimally with no clear process.	Tests the solution partially, missing key cases.	Tests the solution adequately with minor gaps.	Tests the solution thoroughly, covering edge cases.	Tests comprehensively with creative test cases and evaluates results critically.

Table 2
Critical Thinking Rubric

Critical Thinking						
(Evaluating logical reasoning, debugging, and code optimization.)						
Criteria	Bloom's Taxonomy Level	Lacking Proficiency (0)	Emerging (1)	Developing (2)	Proficient (3)	Exceeding Proficiency (4)
Error Identification	Applying	Fails to identify any errors.	Identifies only basic syntax errors.	Identifies syntax and simple logical errors.	Identifies most logical and structural errors effectively.	Identifies complex errors, including hidden logical flaws.
Debugging Strategies	Applying → Evaluating	Attempts debugging randomly without strategy.	Applies basic debugging but lacks consistency.	Uses systematic debugging techniques for common errors.	Debugs effectively using clear and logical steps.	Implements advanced debugging strategies, explaining and justifying all steps.
Optimizing Code	Evaluating	Makes no attempt to optimize code.	Optimizes code minimally for readability or minor efficiency improvements.	Moderately optimizes code for performance.	Optimizes code effectively for efficiency, readability, and maintainability.	Implements advanced optimizations creatively, improving scalability and performance.
Evaluating Solutions	Evaluating	Does not evaluate the solution's effectiveness.	Evaluates the solution only at a basic level.	Evaluates the solution's success for most cases.	Thoroughly evaluates the solution for all requirements.	Evaluates the solution critically, identifying areas for improvement and optimization.
Use of Logical Constructs	Analyzing	Fails to use or understand logical constructs.	Uses logical constructs incorrectly or minimally.	Applies simple logical constructs with moderate success.	Applies logical constructs effectively to create clear, functional logic.	Combines logical constructs creatively to solve complex problems.
Conditional Thinking	Applying → Evaluating	Does not apply logical conditions in programs.	Applies conditions minimally or incorrectly.	Applies conditions correctly but struggles with complexity.	Implements logical conditions effectively in problem-solving.	Implements advanced conditional structures that

(*continued on next page*)

Table 2 (*continued*)

Critical Thinking						
(Evaluating logical reasoning, debugging, and code optimization.)						
Criteria	Bloom's Taxonomy Level	Lacking Proficiency (0)	Emerging (1)	Developing (2)	Proficient (3)	Exceeding Proficiency (4)
						demonstrate clear and flexible thinking.
Application of Patterns	Analyzing → Creating	Does not recognize patterns or apply them.	Recognizes basic patterns but applies them inconsistently.	Uses simple patterns effectively in some cases.	Applies patterns appropriately to improve code structure.	Develops or adapts advanced patterns to create efficient, maintainable solutions.

Table 3
Creativity Rubric

Creativity						
(Assessing originality, innovative thinking, and creative application of programming constructs in programming.)						
Criteria	Bloom's Taxonomy Level	Lacking Proficiency (0)	Emerging (1)	Developing (2)	Proficient (3)	Exceeding Proficiency (4)
Originality of Approach	Creating	Relies entirely on examples without modifications.	Makes minimal modifications to standard examples.	Creates partially original solutions.	Develops original solutions tailored to the problem context.	Produces highly original solutions with innovative approaches and techniques.
Creative Application of Constructs	Creating	Uses constructs in standard, uninspired ways.	Applies constructs with slight creativity.	Combines constructs in somewhat novel ways.	Combines constructs creatively to enhance functionality or problem-solving.	Uses constructs in entirely new and imaginative ways to achieve complex goals.
Exploring Alternatives	Creating	Does not explore any alternative solutions.	Considers a single alternative but does not implement it.	Explores and implements a few alternative approaches.	Evaluates and applies alternatives to improve solutions.	Explores and implements multiple, highly creative alternatives.
Extending Problem Scope	Creating	Sticks rigidly to the defined problem scope.	Adds minor extensions but stays close to the task.	Expands the problem meaningfully with new elements.	Extends the problem significantly with creative additions.	Reimagines the problem entirely, introducing new dimensions or solving related issues.
Application of New Ideas	Creating	Does not attempt to introduce new ideas.	Introduces basic, partially thought-out ideas.	Incorporates moderately new and creative ideas.	Applies new, well-thought-out ideas to enhance the program.	Applies entirely new ideas creatively, redefining the problem or its solution.
Experimentation	Creating	Shows no experimentation or willingness to try new methods.	Tries new methods but with limited success.	Experiments with methods to discover better solutions.	Experiments systematically to improve solutions effectively.	Innovates through experimentation, discovering entirely new ways to solve problems.
Innovative Problem Approach	Analyzing → Creating	Solves problems using basic, direct approaches with no variation.	Solves problems with slight modifications but lacks novelty.	Implements new variations or approaches that show some independent thinking.	Develops innovative approaches that go beyond standard problem-solving methods.	Reframes problems creatively, introducing entirely new or unexpected solutions.

Table 4
Understanding Programming Logic Rubric

Understanding Programming Logic						
(Assessing mastery of programming constructs and logical structures.)						
Criteria	Bloom's Taxonomy Level	Lacking Proficiency (0)	Emerging (1)	Developing (2)	Proficient (3)	Exceeding Proficiency (4)
Use of Variables	Applying	Does not use variables or uses them incorrectly.	Uses variables but lacks meaningful naming or initialization.	Creates meaningful variables but with limited applications.	Creates and uses meaningful variables effectively.	Utilizes variables dynamically and creatively for advanced scenarios.
Use of Loops	Applying	No loops used.	Uses simple loops with fixed iterations.	Implements loops partially (For instance, simple nested loops).	Applies advanced loops effectively (For instance, nested, dynamic).	Combines multiple advanced loops innovatively and efficiently.
Use of Conditionals	Analyzing	No conditionals used or used incorrectly.	Uses basic conditionals but lacks depth (For instance, "if-then").	Implements basic nested or variable-based conditionals.	Applies advanced conditionals (For instance, Boolean expressions).	Implements complex, multi-layered conditionals dynamically and innovatively.

(*continued on next page*)

Table 4 (*continued*)

Understanding Programming Logic						
(Assessing mastery of programming constructs and logical structures.)						
Criteria	Bloom's Taxonomy Level	Lacking Proficiency (0)	Emerging (1)	Developing (2)	Proficient (3)	Exceeding Proficiency (4)
Use of Operators & Expressions	Analyzing	No use of operators	Use of only arithmetic and/or relational operators and no evidence of nested operators in an expression	Use of 1–3 operators, with at least one Boolean operator, or one instance of nested operators in an expression	More than 3 operators, with at least one Boolean operator or one instance of nested operators in an expression	
Program Initialization	Applying	Incorrect initialization or did not attempt to initialize when required	Partially correct initialization or initialization not applicable	Initialization required and correctly done		
Program termination	Evaluating	Program does not terminate	Program terminates			
Code Organization and Clarity	Applying	Code is disorganized and difficult to read.	Code is somewhat organized but lacks clarity.	Code is mostly organized and uses basic comments.	Code is well-organized with clear comments and structure.	Demonstrates exceptional organization with clear comments, naming conventions, and structure.

References

Ahme, Z., Shanto, S. S., & Jony, A. I. (2024). Potentiality of generative AI tools in higher education: Evaluating ChatGPT's viability as a teaching assistant for introductory programming courses. *STEM Education, 4*(3), 165–182. https://doi.org/10.3934/steme.2024011

Aibin, T., Jijun, Y., Wenye, L., Shike, Z., & Dawei, L. (2023). The impact of different types of off-campus training on primary and junior high students' higher-order thinking dispositions. *Thinking Skills and Creativity, 49*. https://doi.org/10.1016/j.tsc.2023.101351

Akgun, S., & Greenhow, C. (2022). Artificial intelligence in education: Addressing ethical challenges in K-12 settings. *AI and Ethics, 2*(3), 431–440. https://doi.org/10.1007/s43681-021-00096-7

Alves, P., & Cipriano, B. P. (2024). "Give me the code"-Log Analysis of First-Year CS Students' interactions with GPT. https://doi.org/10.5281/zenodo.8430808.

Astuti, A. P., Aziz, A., Sumarti, S. S., & Bharati, D. A. L. (2019). Preparing 21st century teachers: Implementation of 4C character's pre-service teacher through teaching practice. In *Journal of physics* (Vol 1233), Article 012109. https://doi.org/10.1088/1742-6596/1233/1/012109, 1.

Basu, S. (2019). Using rubrics integrating design and coding to assess middle school students' open-ended block-based programming projects. *SIGCSE 2019 – Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, 1211–1217. https://doi.org/10.1145/3287324.3287412

Belland, B. R. (2017). *Instructional scaffolding in STEM education strategies and efficacy evidence*. https://doi.org/10.1007/978-3-319-02565-0

Birt, L., Scott, S., Cavers, D., Campbell, C., & Walter, F. (2016). Member checking: A tool to enhance trustworthiness or merely a nod to validation? *Qualitative Health Research, 26*(13), 1802–1811. https://doi.org/10.1177/1049732316654870

Bjursten, E. L., Nilsson, T., & Gumaelius, L. (2023). Computer programming in primary schools: Swedish Technology Teachers' pedagogical strategies. *International Journal of Technology and Design Education, 33*(4), 1345–1368. https://doi.org/10.1007/s10798-022-09786-7

Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology, 3*(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

Brookhart, S. (2010). *How to assess higher order thinking skills in your classroom*. ASCD. http://www.ascd.org/Publications/Books/Overview/How-to-Assess-Higher-Order-Thinking-Skills-in-YourClassroom.aspx.

Cabo, C. (2015). Quantifying student progress through bloom's taxonomy cognitive categories in computer programming courses. *2015 ASEE Annual Conference and Exposition Proceedings*, 26.1295.1–26.1295.13. https://doi.org/10.18260/p.24632

Campbell, D. T., & Stanley, J. C. (1963). *Experimental and quasi-experimental designs for research. Houghton,* Mifflin and Company.

Chandio, M. T., Pandhiani, S. M., & Iqbal, R. (2016). Bloom's taxonomy: Improving assessment and teaching- learning process. *Journal of Education and Educational Development, 3*(2), 203–221. https://jmsnew.iobmre-search.com/index.php/joeed/article/download/194/330.

Chen, Y., Xiao, S., Song, Y., Li, Z., Sun, L., & Chen, L. (2025). MindScratch: A visual programming support tool for classroom learning based on multimodal generative AI. *International Journal of Human-Computer Interaction*, 1–19. https://doi.org/10.1080/10447318.2025.2475991

Dickey, E., Bejarano, A., & Chirayu, G. (2024). AI-Lab: A framework for introducing generative artificial intelligence tools in computer programming courses. *SN Computer. Science., 5*(6), 17. https://doi.org/10.1007/s42979-024-03074-y

Cohen, J. (1992). Statistical power analysis. *Current Directions in Psychological Science, 1*(3), 98–101. https://doi.org/10.1111/1467-8721.ep10768783

Creswell, J. W. (2014). *Research design: Qualitative, quantitative, and mixed methods approaches* (4th ed.). Sage.

Creswell, J. W., & Poth, C. N. (2018). *Qualitative inquiry and research design: Choosing among five approaches*. ISBN: 9781506330204 (4th ed.). SAGE Publications https://us.sagepub.com/en-us/nam/qualitative-inquiry-and-research-design/book246896.

Denny, P., Leinonen, J., Prather, J., Luxton-Reilly, A., Amarouche, T., Becker, B. A., & Reeves, B. N. (2023). Promptly: Using prompt problems to teach learners how to effectively utilize AI code generators. http://arxiv.org/abs/2307.16364.

Deriba, F. G., Sanusi, I. T., & Sunday, A. O. (2023). Enhancing computer programming education using ChatGPT: A mini review. *ACM International Conference Proceeding Series*. https://doi.org/10.1145/3631802.3631848

Devolder, A., van Braak, J., & Tondeur, J. (2012). Supporting self-regulated learning in computer-based learning environments: Systematic review of effects of scaffolding in the domain of science education. *Journal of Computer Assisted Learning, 28*(6), 557–573. https://doi.org/10.1111/j.1365-2729.2011.00476.x

Dimitrov, D. M., & Rumrill, P. D. (2003). Pretest-posttest designs and measurement of change. *Work, 20*(2), 159–165. https://doi.org/10.3233/WOR-2003-00285

Ghimire, A., & Edwards, J. (2024). Coding with AI: How are tools like ChatGPT being used by students in foundational programming courses. In A. M. Olney, I. A. Chounta, Z. Liu, O. C. Santos, & I. I. Bittencourt (Eds.), *Artificial intelligence in education. AIED 2024. Lecture notes in computer science* (Vol. 14830). Cham: Springer. https://doi-org.ezproxy.uef.fi:2443/10.1007/978-3-031-64299-9_20.

Egbert, J., Shahrokni, S. A., Abobaker, R., & Borysenko, N. (2021). It's a chance to make mistakes": Processes and outcomes of coding in 2nd grade classrooms. *Computers & Education, 168*, Article 104173. https://doi.org/10.1016/j.compedu.2021.104173

Erdei, R., Springer, J. A., & Whittinghill, D. M. (2017). An impact comparison of two instructional scaffolding strategies employed in our programming laboratories: Employment of a supplemental teaching assistant versus employment of the pair programming methodology. *IEEE Frontiers in Education Conference (FIE)*, 1–6. https://doi.org/10.1109/FIE.2017.8190650. Indianapolis, IN, USA.

Estevez, J., Garate, G., & Grana, M. (2019). Gentle introduction to artificial intelligence for high-school students using scratch. *IEEE Access, 7*, 179027–179036. https://doi.org/10.1109/ACCESS.2019.2956136

Eteng, I., Akpotuzor, S., Akinola, S. O., & Agbonlahor, I. (2022). A review on effective approach to teaching computer programming to undergraduates in developing countries. *Scientific African, 16*. https://doi.org/10.1016/j.sciaf.2022.e01240

Falloon, G. (2016). An analysis of young students' thinking when completing basic coding tasks using Scratch Jnr. On the iPad. *Journal of Computer Assisted Learning, 32*(6), 576–593. https://doi.org/10.1111/jcal.12155

Fanchamps, N., Slangen, L., Hennissen, P., & Specht, M. (2021). The influence of SRA programming on algorithmic thinking and self-efficacy using Lego robotics in two types of instruction. *International Journal of Technology and Design Education, 31*(2), 203–222. https://doi.org/10.1007/s10798-019-09559-9

Fayzulloeva, G. C., & Mustafoeva, N. U. O. A. (2020). A learner needs analysis report. *International Journal of Innovative Research and Development, 11*, 726. https://doi.org/10.36713/epra2016

Fernandez, A. S., & Cornell, K. A. (2024). CS1 with a side of AI: Teaching software verification for secure code in the era of generative AI. SIGCSE 2024. *Proceedings of the 55th ACM Technical Symposium on Computer Science Education, 1*, 345–351. https://doi.org/10.1145/3626252.3630817

Finnie-Ansley, J., Denny, P., Luxton-Reilly, A., Santos, E. A., Prather, J., & Becker, B. A. (2023). My AI wants to know if this will be on the exam: Testing OpenAI's codex on CS2 programming exercises. In *In proceedings of the 25th australasian computing Education conference (ACE '23)*. ACM. https://doi.org/10.1145/3576123.3576134.

Gendenjamts, S. (2023). Measuring higher-order thinking Skills in science among primary school students using item response theory. *European Journal of Education Studies, 10*(12). https://doi.org/10.46827/ejes.v10i12.5089

González-Calatayud, V., Prendes-Espinosa, P., & Roig-Vila, R. (2021). Artificial intelligence for student assessment: A systematic review. *Applied Sciences, 11*(12), 5467. https://doi.org/10.3390/app11125467, 2021.

Grant, M. M. (2011). Learning, beliefs, and products: Students' perspectives with project-based learning. *Interdisciplinary Journal of Problem-Based Learning, 5*(2), 37–69. https://doi.org/10.7771/1541-5015.1254

Gupta, P., & Mehrotra, D. (2022). Objective assessment in Java programming Language using rubrics. *Journal of Information Technology Education: Innovations in Practice, 21*, 155–173. https://doi.org/10.28945/5040

Hamadeh, L. (2024). Exploring the utilisation of generative AI tools by undergraduate first-year mechanical engineering students in programming assessments. *SEFI Journal of Engineering Education Advancement, 1*(1), 39–52. https://doi.org/10.62492/sefijeea.v1i1.20

Hartley, K., Hayak, M., & Ko, U. H. (2024). Artificial intelligence supporting independent student learning: An evaluative case Study of ChatGPT and learning to code. *Education Sciences, 14*(2). https://doi.org/10.3390/educsci14020120

Hazzan, O., & Erez, Y. (2024). Generative AI in computer science education. In *Proceedings of the 55th ACM technical symposium on computer science education* (Vol. 2, p. 1899) (SIGCSE 2024). Association for Computing Machinery, New York, NY, USA.

Ho, C. L., Liu, X. Y., Qiu, Y. W., & Yang, S. Y. (2024). Research on innovative applications and impacts of using generative AI for user interface design in programming courses. *ACM International Conference Proceeding Series*, 68–72. https://doi.org/10.1145/3658549.3658566

Holmes, W., Porayska-Pomsta, K., Holstein, K., Sutherland, E., Baker, T., Shum, S. B., Santos, O. C., Rodrigo, M. T., Cukurova, M., Bittencourt, I. I., & Koedinger, K. R. (2022). Ethics of AI in education: Towards a community-wide framework. *International Journal of Artificial Intelligence in Education, 32*(3), 504–526. https://doi.org/10.1007/s40593-021-00239-1

Hou, X., Wu, Z., Wang, X., & Ericson, B. J. (2024). CodeTailor: LLM-Powered personalized parsons puzzles for engaging support while learning programming. *L@S 2024 - Proceedings of the 11th ACM Conference on Learning @ Scale*, 51–62. https://doi.org/10.1145/3657604.3662032

Hsu, H. P. (2025). From programming to prompting: Developing computational thinking through large Language model-based generative artificial intelligence. *TechTrends*. https://doi.org/10.1007/s11528-025-01052-6

Huang, Q., Zou, Z., Xing, Z., Zuo, Z., Xu, X., & Lu, Q. (2023). AI chain on large Language model for unsupervised control flow graph generation for statically-typed partial code. http://arxiv.org/abs/2306.00757.

Ibarra-Torres, F., Caiza, G., García, M. V., & Barona-Pico, V. (2024). Use of basic programming tools to foster programming logic in university students with school preparation other than computer science. *Procedia Computer Science, 237*, 413–419. https://doi.org/10.1016/j.procs.2024.05.122

Jackson, V., Vasilescu, B., Russo, D., Ralph, P., Izadi, M., Prikladnicki, R., D'Angelo, S., Inman, S., Lisboa, A., & Van-der H, A. (2024). Creativity, generative AI, and software development. *A Research Agenda*. http://arxiv.org/abs/2406.01966.

Jin, Y., Yang, K., Yan, L., Echeverria, V., Zhao, L., Alfredo, R., Milesi, M., Fan, J., Li, X., Gašević, D., & Martinez-Maldonado, R. (2024). Chatting with a learning analytics dashboard: The role of generative AI literacy on learner interaction with conventional and scaffolding chatbots. http://arxiv.org/abs/2411.15597.

Jonsson, M., & Tholander, J. (2022). Cracking the code: Co-coding with AI in creative programming education. *ACM International Conference Proceeding Series*, 5–14. https://doi.org/10.1145/3527927.3532801

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: A sociocultural perspective. *Computer Science Education, 29*(2–3), 136–176. https://doi.org/10.1080/08993408.2019.1608781

Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. In *In Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)* (pp. 1–23). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3544548.3580919. April 19, 2023, New York, NY, USA.

Kiesler, N. (2022). Reviewing constructivist theories to help foster creativity in programming education. *IEEE Frontiers in Education Conference (FIE)*, 1–5. https://doi.org/10.1109/FIE56618.2022.9962699. Uppsala, Sweden.

Kimmel, B., Lee Geisert, A., Yaro, L., Gipson, B., Taylor Hotchkiss, R., Kwame Osae-Asante, S., Vaught, H., Wininger, G., & Yamaguchi, C. (2024). Enhancing programming error messages in real time with generative AI. *Conference on Human Factors in Computing Systems - Proceedings*. https://doi.org/10.1145/3613905.3647967

Korkmaz, O., Çakir, R., & Ozden, M. Y. (2017). A validity and reliability study of the computational thinking scales (CTS). *Computers in Human Behavior, 72*, 558–569. https://doi.org/10.1016/j.chb.2017.01.005

Korstjens, I., & Moser, A. (2018). Series: Practical guidance to qualitative research. Part 4: Trustworthiness and publishing. *The European Journal of General Practice, 24*(1), 120–124. https://doi.org/10.1080/13814788.2017.1375092

Koutcheme, C., Sarsa, S., Leinonen, J., Hellas, A., & Denny, P. (2023). Automated Program repair using generative models for code infilling. *Springer Nature Switzerland*, 798–803. https://doi.org/10.1007/978-3-031-36272-9_74

Krathwohl, D. R. (2002). A revision of Bloom's taxonomy: An overview. *Theory into Practice, 41*(4), 212–218. https://doi.org/10.1207/s15430421tip4104_2

Kurtz, G., Amzalag, M., Shaked, N., Zaguri, Y., Kohen-Vacs, D., Gal, E., Zailer, G., & Barak-Medina, E. (2024). Strategies for integrating generative AI into higher education: Navigating challenges and leveraging opportunities. *Education Sciences, 14*(5). https://doi.org/10.3390/educsci14050503

Lee, C., Myung, J., Han, J., Jin, J., & Oh, A. (2024). Learning from teaching assistants to formulate subgoals for programming tasks: Exploring the potential for AI teaching assistants. *CEUR Workshop Proceedings, 3840*. https://ceur-ws.org/Vol-3840/L3MNGET24_paper5.pdf.

Leinonen, J., Denny, P., MacNeil, S., Sarsa, S., Bernstein, S., Kim, J., Tran, A., & Hellas, A. (2023). Comparing code explanations created by students and large Language models. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1 (Turku, Finland) (ITiCSE 2023)* (pp. 124–130). *New York*, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3587102.3588785.

Lewis, C. (2022). Automatic programming and education. In *Companion proceedings of the 6th international conference on the art, science, and engineering of programming (programming '22)* (pp. 70–80). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3532512.3539664.

Liao, J., Zhong, L., Zhe, L., Xu, H., Liu, M., & Xie, T. (2024). Scaffolding computational thinking with ChatGPT. *IEEE Transactions on Learning Technologies, 17*, 1668–1682. https://doi.org/10.1109/TLT.2024.3392896

Logacheva, E., Hellas, A., Prather, J., Sarsa, S., & Leinonen, J. (2024). Evaluating contextually personalized programming exercises created with generative AI. ICER 2024. *ACM Conference on International Computing Education Research, 1*, 95–113. https://doi.org/10.1145/3632620.3671103

Luna, S. C. (2015). The futures of learning 2: What kind of learning for the 21st century?. https://www.dpsgs.org/pdf/The_Futures_of_Learning.pdf. M, L. (2003). Thinking in education. Cambridge.

MacNeil, S., Tran, A., Hellas, A., Kim, J., Sarsa, S., Denny, P., Bernstein, S., & Leinonen, J. (2023). Experiences from using code explanations generated by large language models in a web software development e-book. In *Proceedings of the 54th ACM technical symposium on computer science education* (Vol. 1, pp. 931–937). https://doi.org/10.1145/3545945.3569785

MacNeil, S., Tran, A., Mogil, D., Bernstein, S., Ross, E., & Ziheng, H. Z. (2022). Generating diverse code explanations using the gpt-3 large language model. In *Proceedings of the 2022 ACM conference on international computing Education research* (Vol. 2, pp. 37–39). https://doi.org/10.1145/3501709.3544280. New York, NY, USA.

Malik, S. I. (2019). Assessing the teaching and learning process of an introductory programming course with bloom's taxonomy and Assurance of Learning (AOL). *International Journal of Information and Communication Technology Education, 15*(2), 130–145. https://doi.org/10.4018/IJICTE.2019040108

Maryani, I., Prasetyo, Z. K., Wilujeng, I., Purwanti, S., & Fitrianawati, M. (2021). Higher-Order thinking skills multiple choice and essay questions: A validated instrument to measure higher-order thinking skills of prospective teachers. *Journal of Turkish Science Education, 18*(4), 674–690. https://doi.org/10.36681/tused.2021.97

Min, L., & Heng, Z. (2024). A Study on impact of junior high school students' programming learning effect based on generative artificial intelligence. *International Conference on Educational Technology (ICET). Asia Pacific Journal of Education, 44*(1), 1–27. https://doi.org/10.1080/02188791.2024.2305161

Misrom, N. S., Abdurrahman, M. S., Abdullah, A. H., Osman, S., Hamzah, M. H., & Fauzan, A. (2020). Enhancing students' Higher-Order Thinking Skills (HOTS) through an inductive reasoning strategy using geogebra. *International Journal of Emerging Technologies in Learning, 15*(3), 156–179. https://doi.org/10.3991/ijet.v15i03.9839

Naeem, U., Virkki, O. T., & Styve, A. (2024). Developing critical thinking practices interwoven with generative AI usage in an introductory programming course. *2024 IEEE global engineering education conference (EDUCON)*, Article 10578746. https://doi.org/10.1109/educon60312.2024, 2024.

Nathaniel, J., Oyelere, S. S., Suhonen, J., & Tedre, M. (2025). Literature review on the integration of generative AI in programming education. *International Journal of Artificial Intelligence in Education*.

Nowell, L. S., Norris, J. M., White, D. E., & Moules, N. J. (2017). Thematic analysis: Striving to meet the trustworthiness criteria. *International Journal of Qualitative Methods, 16*(1), 1–13. https://doi.org/10.1177/1609406917733847

Obaido, G., Agbo, F. J., Alvarado, C., & Oyelere, S. S. Y. (2023). Analysis of attrition studies within the Computer Sciences. *IEEE Access, 11*, 53736–53748. https://doi.org/10.1109/ACCESS.2023.3280075

Omer, W. (2025). Modified bloom's taxonomy: Perspective of a doctor of philosophy scholar. *Journal of Rawalpindi Medical College, 28*(4). https://doi.org/10.37939/jrmc.v28i4.2804

Oyelere, S. S., Suhonen, J., & Laine, T. (2017). Integrating parson's programming puzzles into a game-based mobile learning application. In *Proceedings of the 17th Koli calling international conference on computing education research*. https://doi.org/10.1145/3141880.3141882

Papadakis, S. (2020). Evaluating a game-development approach to teach introductory programming concepts in secondary education. *International Journal of Technology Enhanced Learning, 12*(2), 127–145. https://doi.org/10.1504/ijtel.2020.106282

Poitras, E., Crane, B., & Siegel, A. (2024). Generative AI in introductory programming instruction: Examining the assistance dilemma with LLM-Based code generators. SIGCSE virtual 2024. *Proceedings of the 2024 ACM Virtual Global Computing Education Conference, 1*, 186–192. https://doi.org/10.1145/3649165.3690111

Prasad, P., & Sane, A. (2024). A self-regulated learning framework using generative AI and its application in CS educational intervention design. SIGCSE 2024. *Proceedings*

of the 55th ACM Technical Symposium on Computer Science Education, 1, 1070–1076. https://doi.org/10.1145/3626252.3630828

Rahe, C., & Maalej, W. (2025). How do programming students use generative AI?. https://doi.org/10.1145/3715762.

Ramona, M., & Aleksandar, M. (2025). Effective prompt engineering for generative AI in C++ programming tasks. *World Journal of Advanced Research and Reviews, 25*(2), 1390–1397. https://doi.org/10.30574/wjarr.2025.25.2.0516

Roth, A., Singh, G., & Turnbow, D. (2021). Equitable but not diverse: Universal design for learning is not enough. *Open access*. Publications from the University of California. https://www.inthelibrarywiththeleadpipe.org/2021/equitable-but-not-diverse/.

Rozita, K., Saiful, N. W., & Mohd, S. B. (2021). Students' assessments in learning programming based on bloom's taxonomy. *Journal of Computing Research and Innovation (JCRINN), 6*(3), 13–21, 2021 https://jcrinn.com:eISSN:2600-8793.

Santos, E. A., Prasad, P., & Becker, B. A. (2023). Always provide context: The effects of code context on programming error message enhancement. In *Proceedings of the ACM conference on global computing education* (Vol. 1, pp. 147–153). https://doi.org/10.1145/3576882.3617909. New York, NY, USA.

Saunders, B., Sim, J., Kingstone, T., Baker, S., Waterfield, J., Bartlam, B., & Jinks, C. (2018). Saturation in qualitative research: Exploring its conceptualization and operationalization. *Quality and Quantity, 52*, 1893–1907. https://doi.org/10.1007/s11135-017-0574-8

Sauvola, J., Tarkoma, S., Klemettinen, M., Riekki, J., & Doermann, D. (2024). Future of software development with generative AI. *Automated Software Engineering, 31*(1). https://doi.org/10.1007/s10515-024-00426-z

Schefer-Wenzl, S., Vogl, C., Peiris, S., & Miladinovic, I. (2025). Exploring the adoption of generative AI tools in computer Science Education: A Student Survey. *Proceedings of the 2024 the 16th international conference on education technology and computers*. ICETC. https://doi.org/10.1145/3702163.3702188, 2024.

Shabani, K., Khatib, M., & Ebadi, S. (2010). Vygotsky's Zone of proximal development: Instructional implications and teachers' professional development. *English Language Teaching, 3*(4). https://doi.org/10.5539/elt.v3n4p237

Shanto, S. S., Jony, A. I., & Ahmed, Z. (2024). Potentiality of generative AI tools in higher education: Evaluating chatgpt's viability as a teaching assistant for introductory programming courses. *STEM education, 4*(3).

Sobral, S. R. (2021). Bloom's taxonomy to improve teaching-learning in introduction to programming. *International journal of information and education technology. IJIET 2021, 11*(3), 148–153. https://doi.org/10.18178/ijiet.2021.11.3.1504. ISSN: 2010-3689.

Spinellis, D. (2024). Pair programming with generative AI. *IEEE Software, 41*(3), 16–18. https://doi.org/10.1109/MS.2024.3363848

Stone, I. (2024). Exploring human-centered approaches in generative AI and introductory programming research. *A scoping review proceedings of the 2024 conference on United Kingdom & Ireland computing education research* (pp. 1–7). New York, NY, USA: (UKICER '24). Association for Computing Machinery. Article 4 https://doi.org/10.1145/3689535.3689553.

Strawhacker, A., Lee, M., & Bers, M. U. (2018). Teaching tools, teachers' rules: Exploring the impact of teaching styles on young children's programming knowledge in ScratchJr. *International Journal of Technology and Design Education, 28*(2), 347–376. https://doi.org/10.1007/s10798-017-9400-9

Sullivan, R. F. (2017). Constructionism. *Open & distance education and eLearning: Creativity, technology, and learning.* eBook ISBN9781315765143.

Sun, H., Xie, Y., & Lavonen, J. (2022). Exploring the structure of students' scientific higher order thinking in science education. *Thinking Skills and Creativity, 43*. https://doi.org/10.1016/j.tsc.2022.100999

Sunday, K., Ocheja, P., Hussain, S., Oyelere, S. S., Balogun, O. S., & Agbo, F. J. (2020). Analyzing student performance in programming education using classification techniques. *International Journal of Emerging Technologies in Learning, 15*(2), 127–144. https://doi.org/10.3991/ijet.v15i02.11527

Ullah, Z., Lajis, A., Jamjoom, M., Altalhi, A., & Saleem, F. (2020). Bloom's taxonomy: A beneficial tool for learning and assessing students' competency levels in computer programming using empirical analysis. *Computer Applications in Engineering Education, 28*(6), 1628–1640. https://doi.org/10.1002/cae.22339

Viswanathan, P. S. (2025). Prompt engineering for conversational AI systems: A systematic review of techniques and applications. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 11*(1), 733–741. https://doi.org/10.32628/CSEIT25111276

Walter, Y. (2024). Embracing the future of Artificial Intelligence in the classroom: The relevance of AI literacy, prompt engineering, and critical thinking in modern education. *International Journal of Educational Technology in Higher Education, 21*(1). https://doi.org/10.1186/s41239-024-00448-3

Wermelinger, M. (2023). Using GitHub copilot to solve simple programming problems. In *In proceedings of the 54th ACM technical symposium on computer science EducationV.1 (SIGCSE 2023)*. ACM. https://doi.org/10.1145/3545945.3569830.

Xue, Y., Chen, H., Bai, G. R., Tairas, R., & Huang, Y. (2024). Does ChatGPT help with introductory programming? An experiment of students using ChatGPT in CS1. *Proceedings - International Conference on Software Engineering*, 331–341. https://doi.org/10.1145/3639474.3640076

Yan, W., Nakajima, T., & Sawada, R. (2024). Benefits and challenges of Collaboration between students and conversational generative artificial intelligence in programming learning: An empirical case Study. *Education Sciences, 14*(4). https://doi.org/10.3390/educsci14040433

Yatigammana, K., Premkumar, P. P., Yatigammana, M., & Kannangara, S. (2023). Impact of generative ai on critical thinking skills in undergraduates: A systematic review impact of generative ai on critical thinking skills in undergraduates: A systematic review the Journal of Desk Research Review and Analysis. In *The journal of desk Research Review and Analysis-JDRRA* (Vol. 1). https://orcid.org/0009-0005-1950-190X.

Yilmaz, R., & Yilmaz, K. F. G. (2023). The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. *Computers and Education: Artificial Intelligence, 4*. https://doi.org/10.1016/j.caeai.2023.100147

Zavala, L., & Mendoza, B. (2018). On the use of semantic-based AIG to automatically generate programming exercises. In *Proceedings of the 49th ACM technical symposium on computer science education (SIGCSE '18)* (pp. 14–19). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3159450.3159608.

Zhou, Y., Gan, L., Chen, J., Wijaya, T. T., & Li, Y. (2023). Development and validation of a Higher-Order Thinking Skills assessment scale for pre-service teachers. *Thinking Skills and Creativity, 48*. https://doi.org/10.1016/j.tsc.2023.101272

Zviel-Girshin, R. (2024). The good and bad of AI tools in novice programming education. *Education Sciences, 14*(10). https://doi.org/10.3390/educsci14101089